UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

Ing. Tecnologías de la Información e Innovación Digital

Asignatura: Optativa I. Bases de Datos en la Nube

Profesor: M.T.I. Marco Antonio Ramírez Hernández

Grado y Grupo: 7° B

PROYECTO

Práctica 03 — Manipulación de datos, relación N:M, procedimientos, funciones, eventos y vista

---

Presentado por:

1. **Osmar Gael Ortega Lucas** — Matricula: `240036`

1. Objetivo

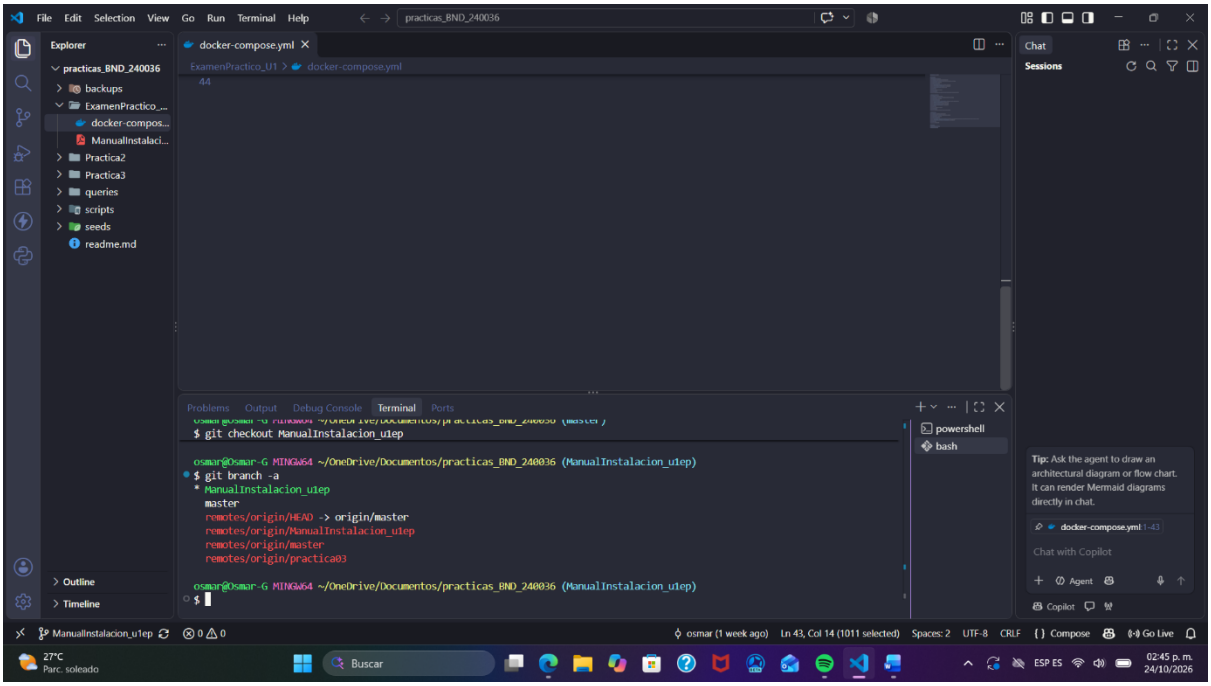
Documentar el procedimiento completo para configurar localmente un entorno con **PostgreSQL 15** y **pgAdmin 4** en Visual Studio Code, subir la configuración a **GitHub** mediante un *Personal Access Token (PAT)*, conectarse a un servidor **VPS** mediante **PuTTY** (o terminal) y desplegar los contenedores para acceder a la base de datos a través de la interfaz web.

2. Configuración Local del Proyecto en Visual Studio Code

1. **Creación de la carpeta del proyecto:**
   Abre Visual Studio Code y selecciona **File > Open Folder**. Crea y selecciona una carpeta para el proyecto (por ejemplo, docker-compose-host).

   En nuestro caso no hicimos eso, porque ya tenemos un repositorio al cual solo se le creo una nueva rama llamada ManualInstalacion_u1ep, como se muestra en la siguiente captura de pantalla por medio del comando

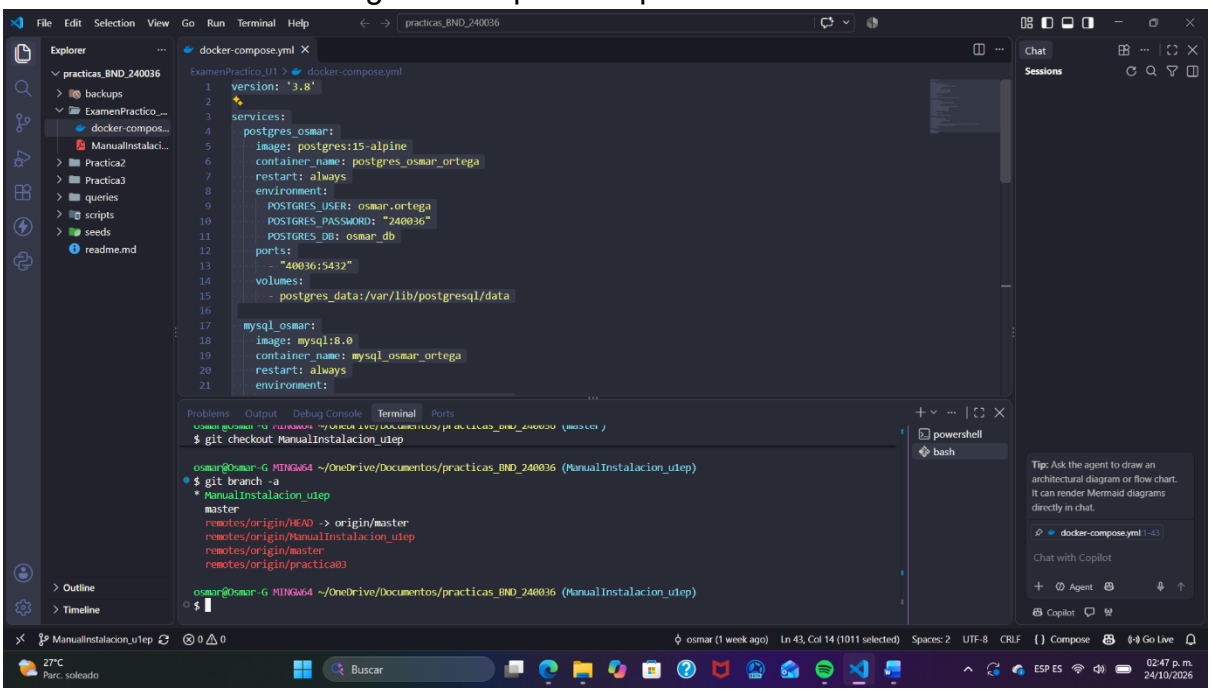
   git branch -a:



2. **Creación del archivo docker-compose.yml:**
   Crea un nuevo archivo en la raíz del proyecto nombrado exactamente docker-compose.yml.

   Nuevamente en nuestro caso particular tenemos la rama antes mencionada, en la cual dentro de la misma tenemos que crear dos archivos dentro de una carpeta, la carpeta se llama ExamenPractico_U1, y dentro de esa carpeta se crean los dos archivos mencionados anteriormente los cuales son, docker-compose.yml (este es el entorno donde se configura la imagen) y manualInstalacion.pdf (este es el pdf del Manual de instalación el cual es este).

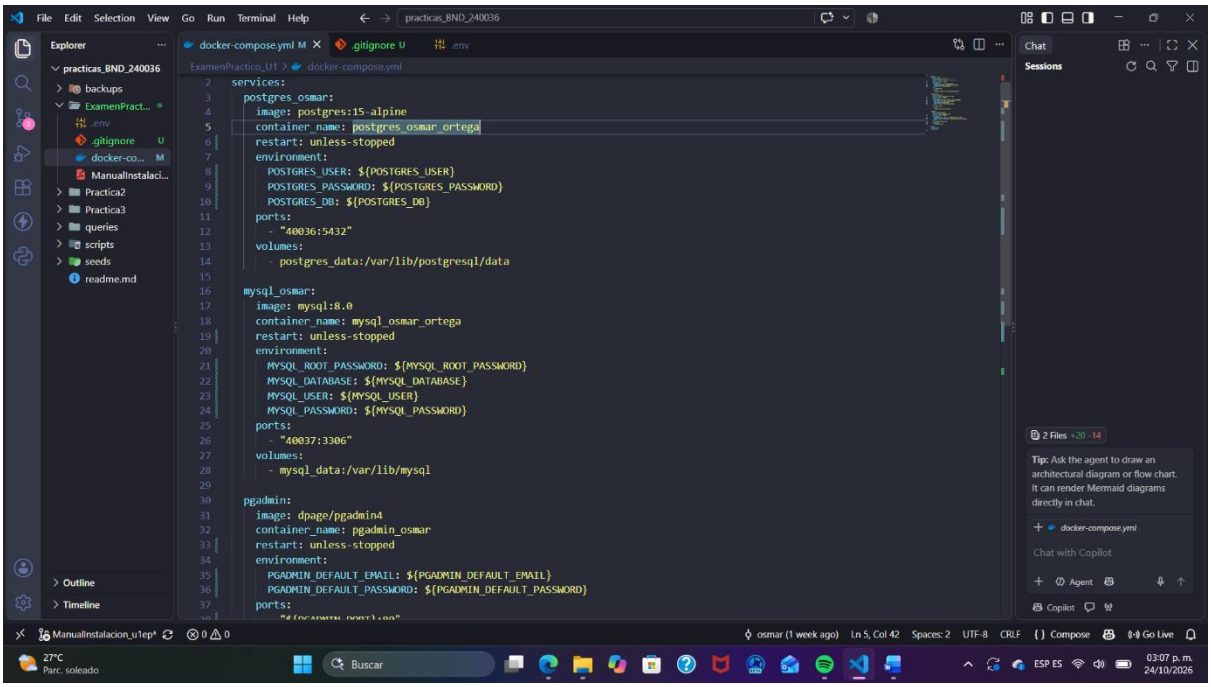
   Como se muestra en la siguiente captura de pantalla:

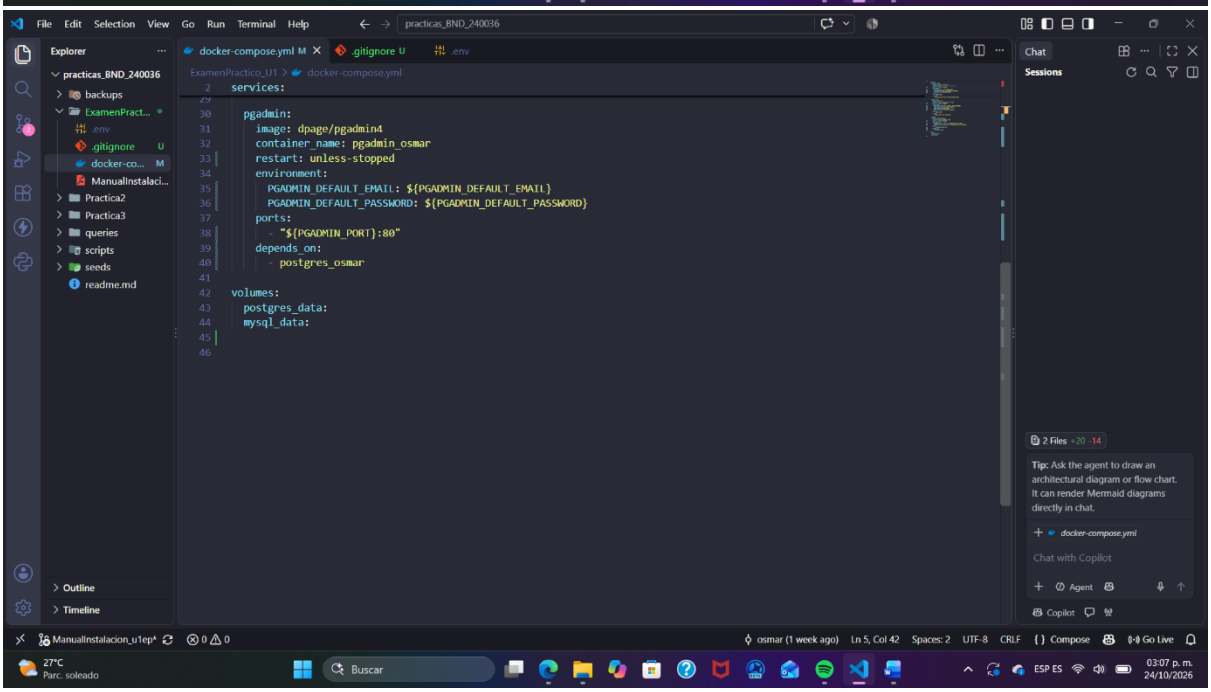


3. **Estructura de la configuración:**
   Agrega la siguiente definición de servicios (reemplaza `240036` por tu matrícula y `40036` por el puerto asignado)

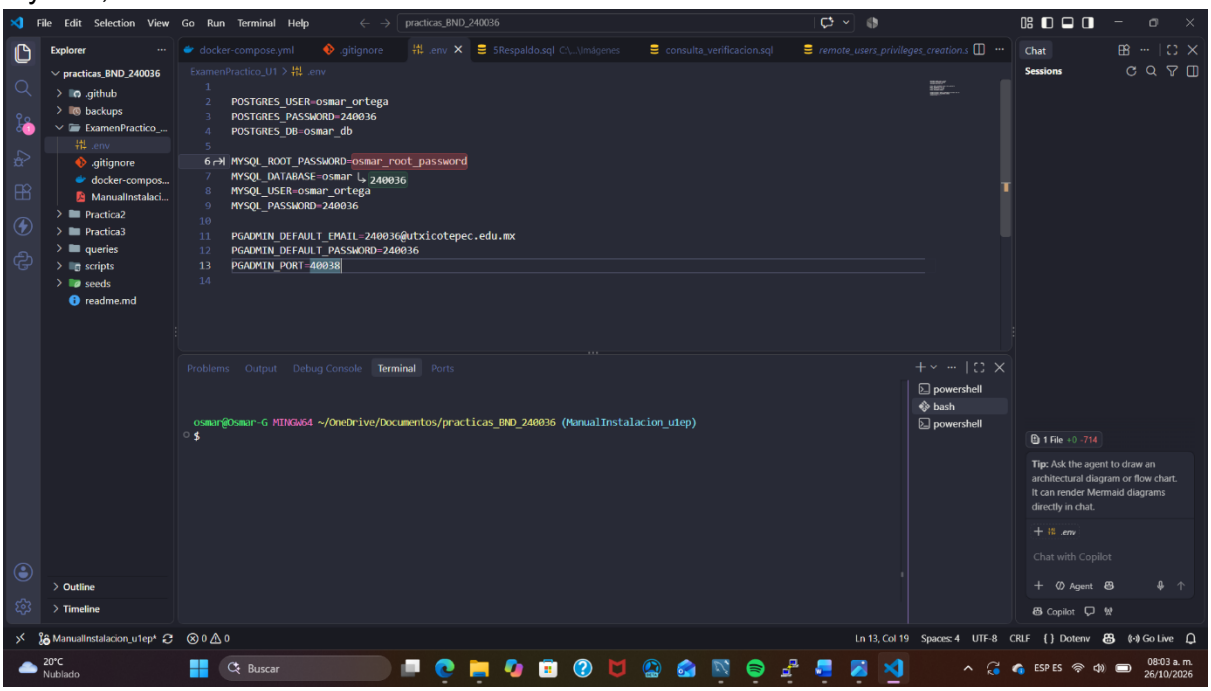
   En el archivo `docker-compose.yml`, definí la configuración de los tres servicios requeridos (**PostgreSQL 15**, **MySQL 8.0** y **pgAdmin 4**), adaptando mis parámetros personales con mi usuario `osmar`, mi matrícula `240036` y los puertos que me fueron asignados (`40036` para PostgreSQL, `40037` para MySQL y para pgAdmin):

3.1. Código completo de mi archivo `docker-compose.yml`

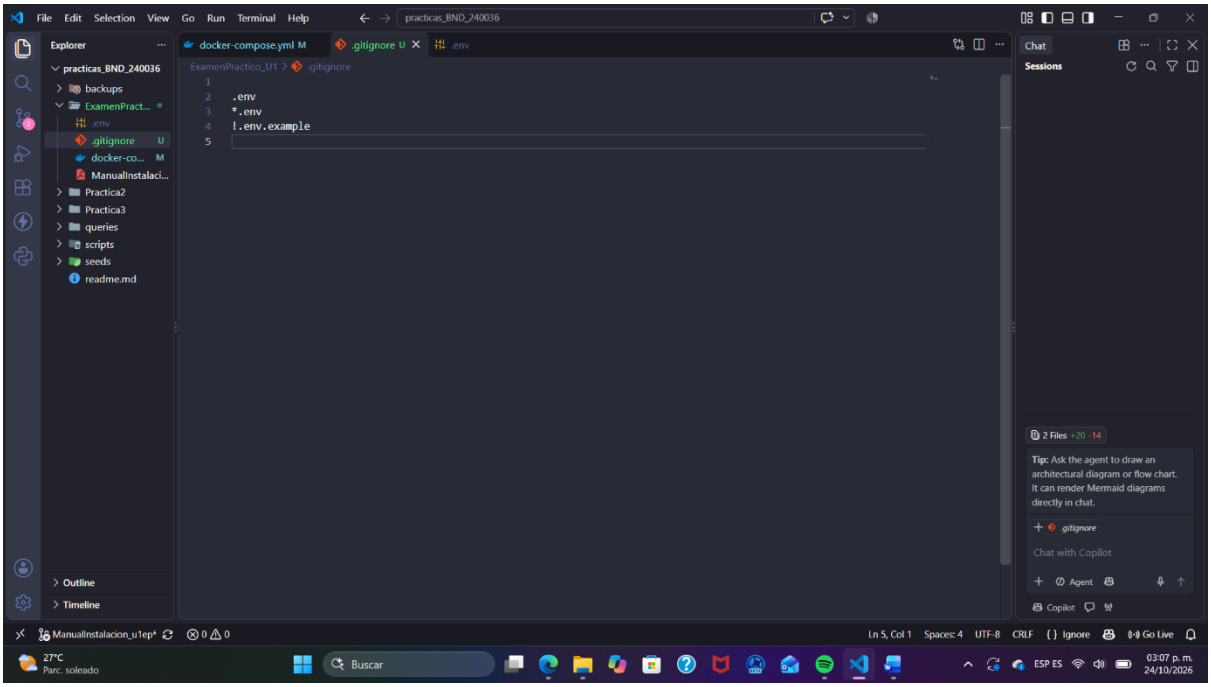




3.2 Se creo un .env en donde solo se almacena los nombres y las contraseñas de MySQL,POSTGRES Y PGADMIN



3.3 Se creo un archivo .gitignore por motivos de seguridad para que no estén disponibles las credenciales



3.2. Explicación detallada de los parámetros por servicio

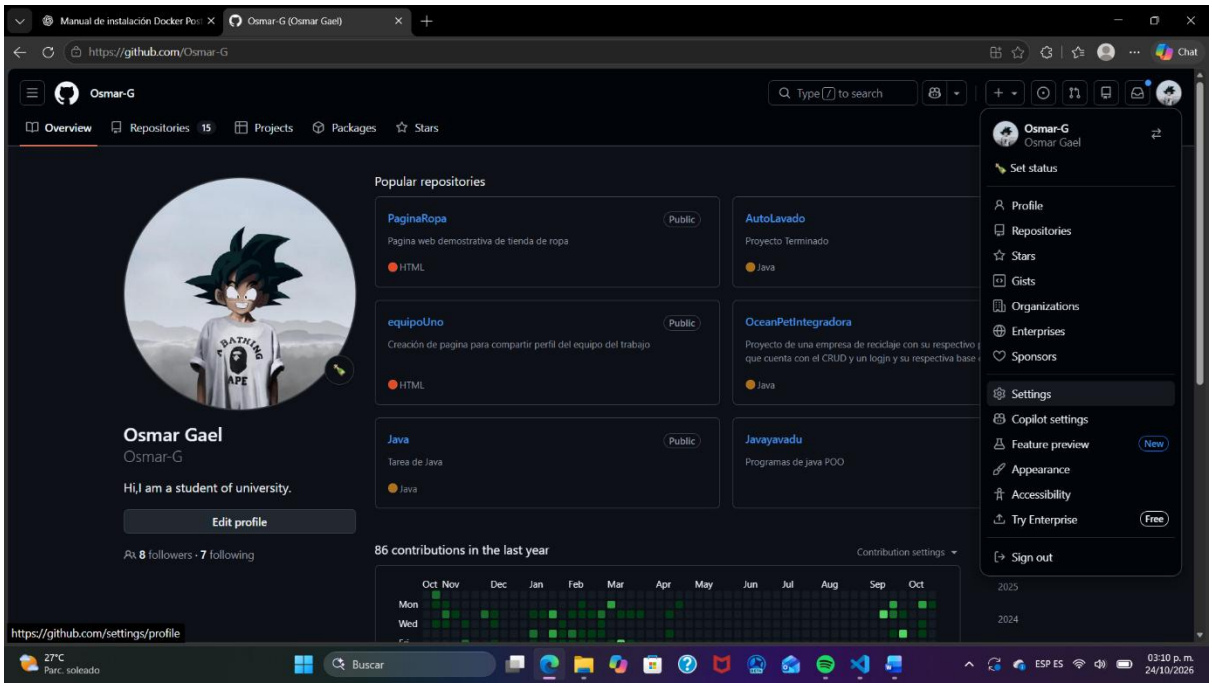
- **Servicio de PostgreSQL (`postgres_osmar`):**
  - **`image: postgres:15-alpine`**: Indiqué que se descargue la versión 15 de PostgreSQL basada en la distribución liviana Alpine Linux.
  - **`container_name: postgres_osmar`**: Asigné un nombre personalizado al contenedor para identificarlo fácilmente en Docker.
  - **`restart: always`**: Establecí que el contenedor se mantenga encendido o se reinicie automáticamente ante caídas del sistema o del servidor.
  - **`environment`**:
    - `POSTGRES_USER: osmar`: Asigné mi nombre como el superusuario de la base de datos.
    - `POSTGRES_PASSWORD: "240036"`: Establecí mi matrícula como contraseña de acceso.
    - `POSTGRES_DB: osmar_db`: Definí el nombre de la base de datos que se creará por defecto al iniciar el servicio.
  - **`ports` (`"40036:5432"`)**: Enlacé el puerto `40036` de mi VPS con el puerto interno `5432` del contenedor de PostgreSQL.
  - **`volumes`**: Enlacé el volumen persistente `postgres_data_240036`con la ruta `/var/lib/postgresql/data` dentro del contenedor para no perder la información guardada.
- **Servicio de MySQL (`mysql_osmar`):**
  - **`image: mysql:8.0`**: Seleccioné la imagen oficial del motor de base de datos MySQL en su versión 8.0.
  - **`container_name: mysql_osmar`**: Definí el nombre del contenedor correspondiente al motor MySQL.
  - **`restart: always`**: Configuré la política para mantener activo el servicio continuamente.
  - **`environment`**:
    - `MYSQL_ROOT_PASSWORD: osmar_root_password`: Asigné la contraseña para el usuario administrador supremo (`root`).
    - `MYSQL_DATABASE: osmar_db`: Indiqué la base de datos a instanciar al iniciar el contenedor.
    - `MYSQL_USER: osmar` y `MYSQL_PASSWORD: "240036"`: Creé mi usuario de trabajo individual vinculado directamente a mi base de datos `osmar_db`.

    - **`ports` (`"40037:3306"`)**: Redireccioné el puerto público `40037` asignado en el servidor hacia el puerto estándar `3306` interno de MySQL.
    - **`volumes`**: Asocié el volumen `mysql_data_osmar` con la ruta interna `/var/lib/mysql`.
- **Servicio de la Interfaz Web (`pgadmin`):**
    - **`image: dpage/pgadmin4`**: Declaré la imagen oficial del administrador gráfico pgAdmin 4.
    - **`container_name: pgadmin_global`**: Asigné el nombre identificador para el contenedor de la interfaz de administración.
    - **`restart: always`**: Aseguré el reinicio automático del servicio.
    - **`environment`**:
        - `PGADMIN_DEFAULT_EMAIL: "240036@utxicotepec.edu.mx"`: Configuré mi correo institucional como nombre de usuario para el inicio de sesión.
        - `PGADMIN_DEFAULT_PASSWORD: "240036"`: Establecí mi matrícula como contraseña para ingresar a la plataforma web.
    - **`ports` (`"40038:80"`)**: Mapeé directamente el puerto HTTP estándar `40038` del servidor con el puerto `80` del contenedor web.
- **Sección de Volúmenes (`volumes`):**
    - **`postgres_data_osmar`** y **`mysql_data_osmar`**: Definí ambos volúmenes locales independientes para garantizar que la información de mis bases de datos persista dentro del disco del servidor, aun si elimino o vuelvo a levantar los contenedores.

4. Generación del Personal Access Token (PAT) en GitHub

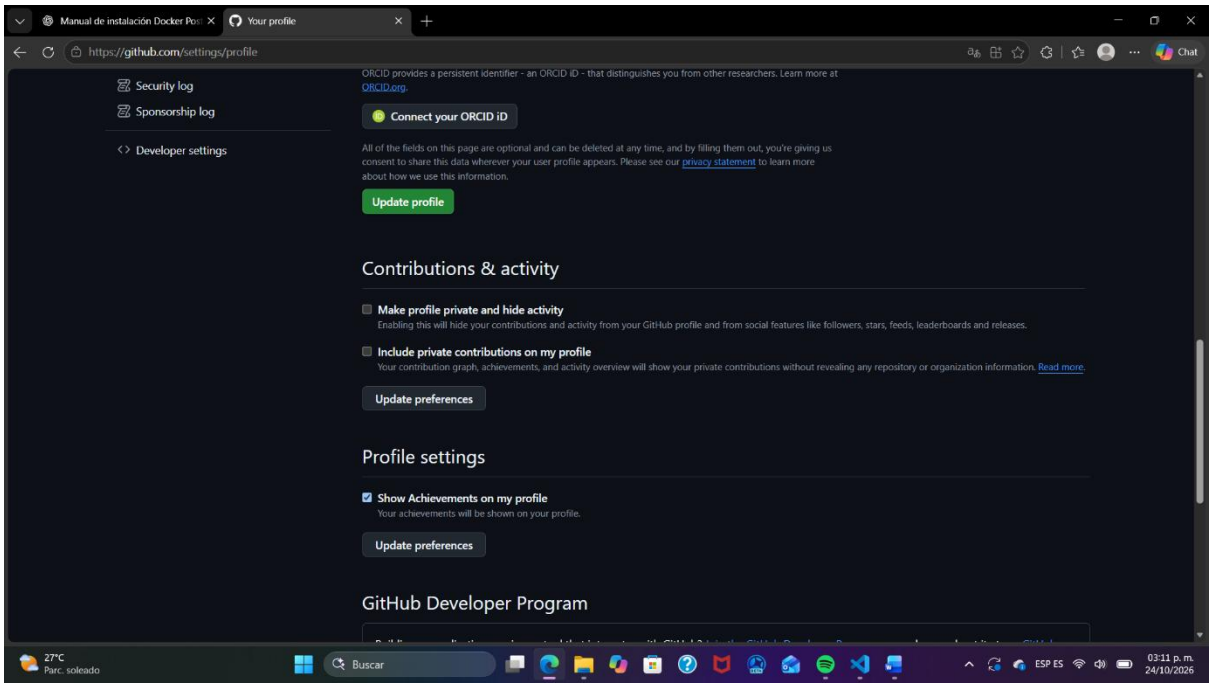
Dado que GitHub requiere autenticación mediante tokens de acceso personal para ejecutar comandos de Git desde la terminal:

1. **Acceso a la configuración de la cuenta:**
   Inicié sesión en mi cuenta de GitHub, hice clic en mi foto de perfil (esquina superior derecha) y seleccioné **Settings**.

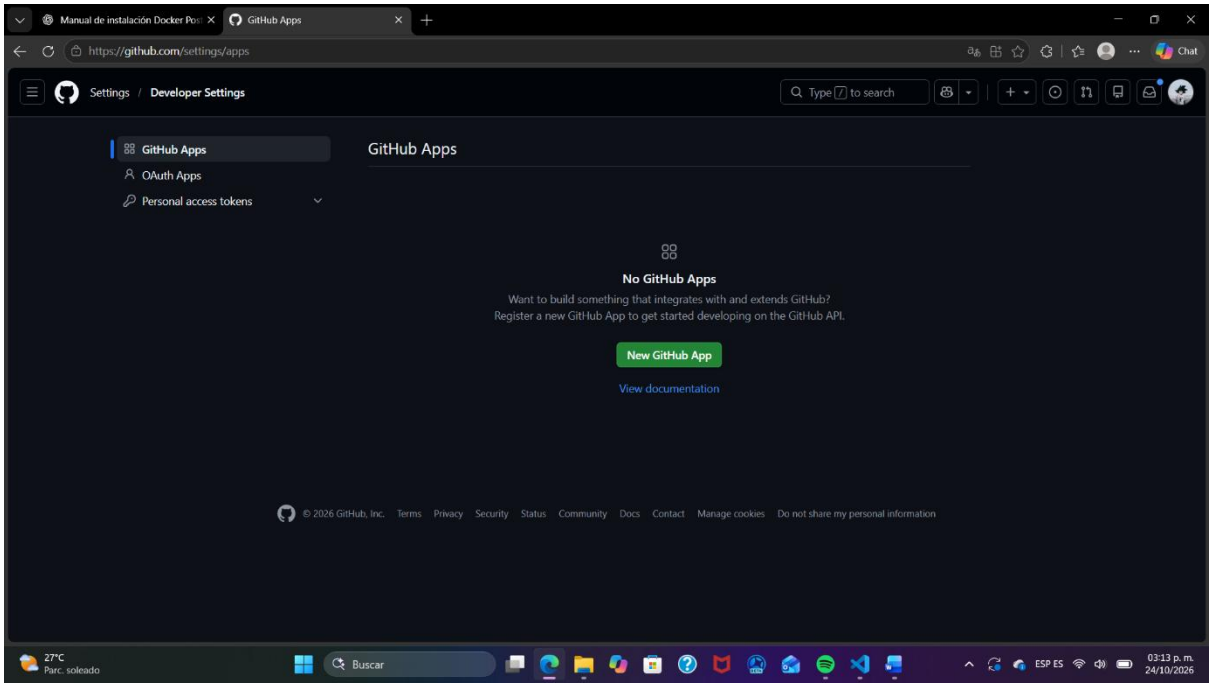


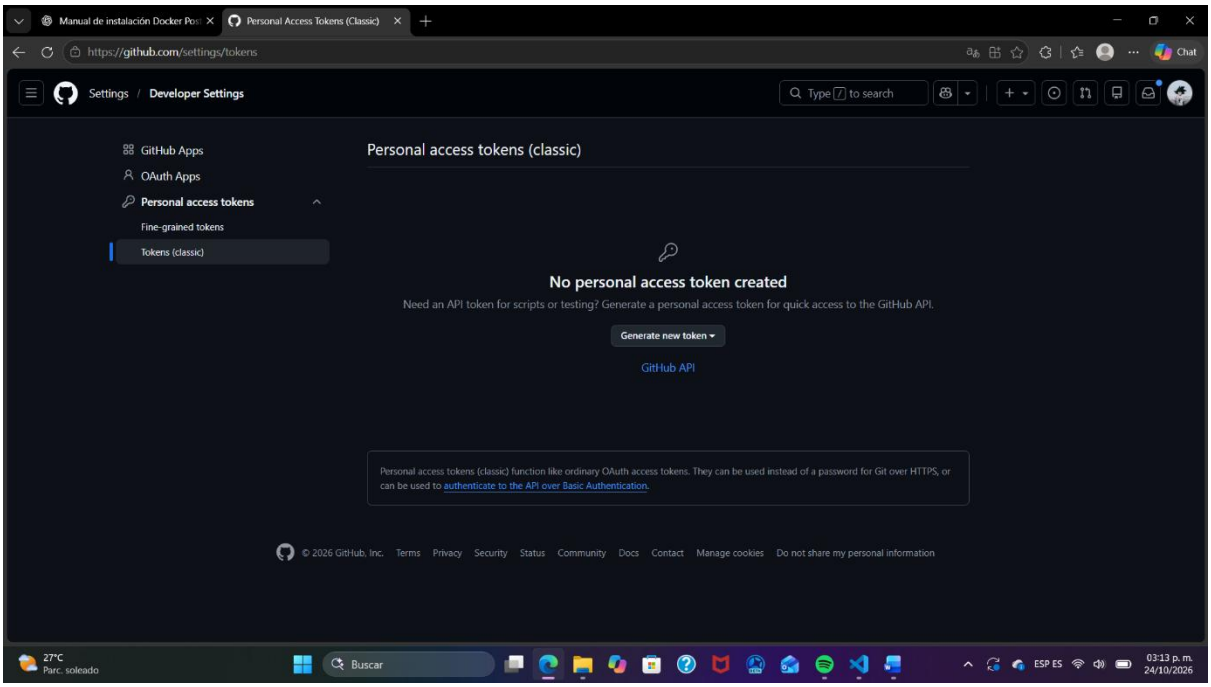
2.

3. **Ingreso a los ajustes de desarrollador:**
   En el menú lateral izquierdo, me desplacé hacia la parte inferior y seleccioné **Developer settings**.

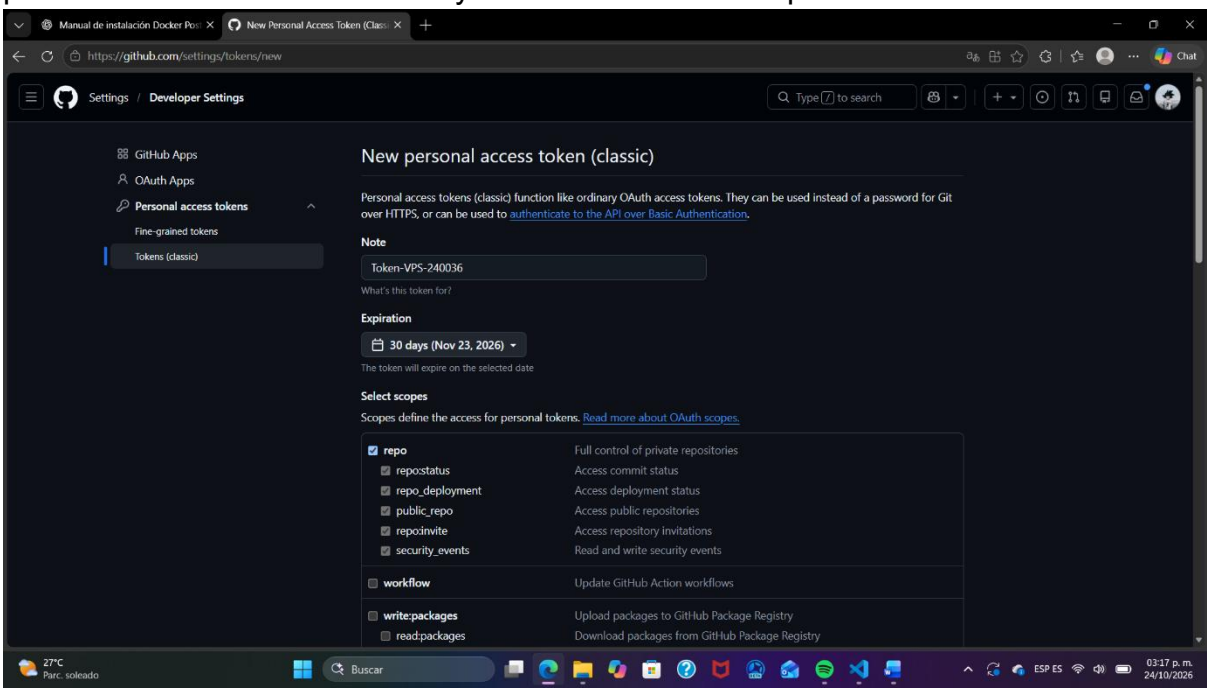


4. **Creación del Token Classic:**
   Navegué a **Personal access tokens > Tokens (classic)** y seleccioné la opción **Generate new token > Generate new token (classic)**

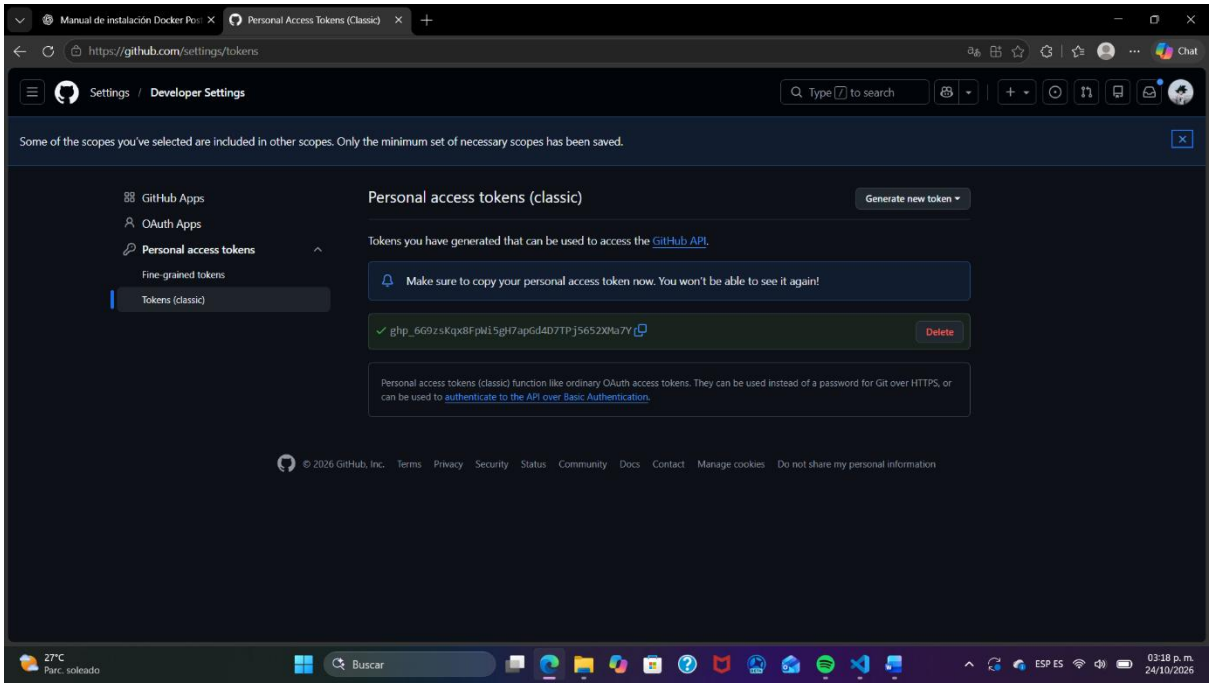




5. **Configuración de los parámetros del token:**
   Completé los campos de la siguiente manera:
   - **Note:** Asigné un identificador para reconocer el token (por ejemplo: Token-VPS-240036).
   - **Expiration:** Seleccioné el tiempo de validez del token (30 días o sin expiración).
   - **Select scopes:** Marqué la casilla principal **repo** para otorgar los permisos necesarios de lectura y escritura sobre mis repositorios.



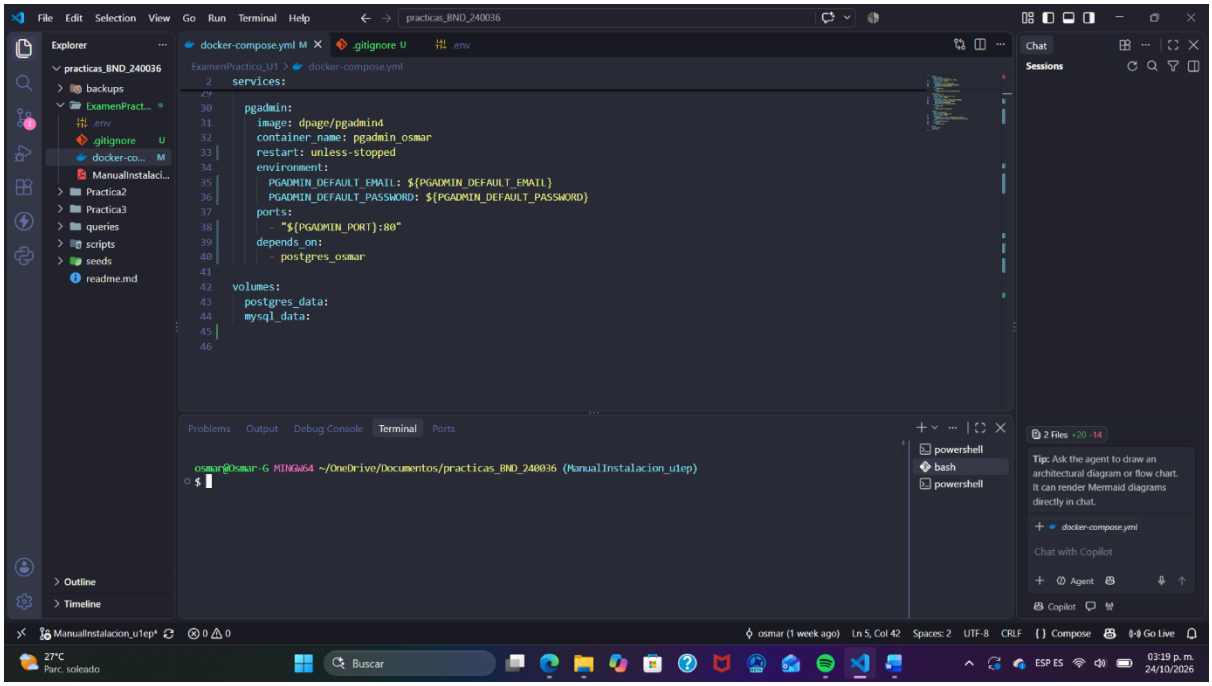
6. **Generación y resguardo del token:**
   Presioné el botón **Generate token** al final del formulario. Copié la clave generada que comienza con ghp_... y la guardé en un lugar seguro, ya que este token es la contraseña que utilizo al ejecutar los comandos.



5. Subir el Proyecto a mi Repositorio de GitHub

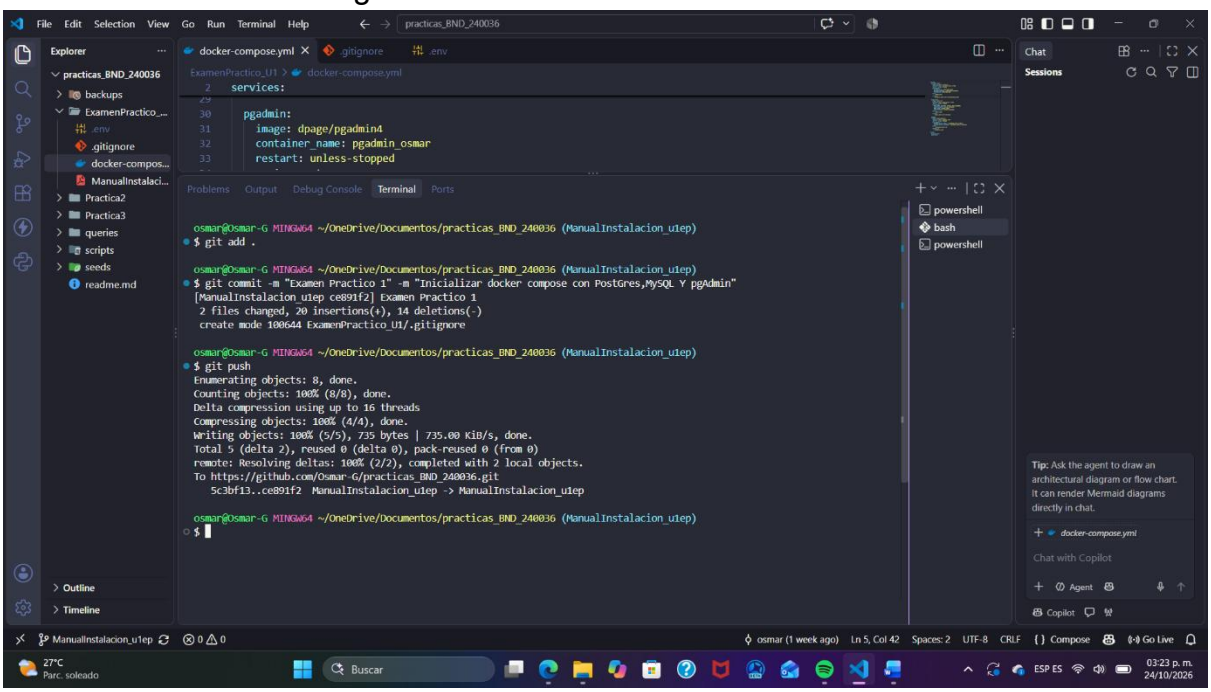
Para respaldar mi archivo docker-compose.yml en el repositorio que ya tenía creado previamente en GitHub y poder descargarlo posteriormente en el servidor VPS:

1. **Apertura de la terminal integrada:**
   De regreso en Visual Studio Code, abrí la terminal integrada presionando la combinación de teclas **Ctrl + Ñ** (o desde el menú **Terminal > New Terminal**).



2. **Vinculación y envío de comandos mediante Git:**
   Ejecuté uno a uno la siguiente serie de comandos para inicializar el repositorio local, agregar mi archivo de configuración, realizar el commit y subir los cambios a mi repositorio existente docker-compose-host:

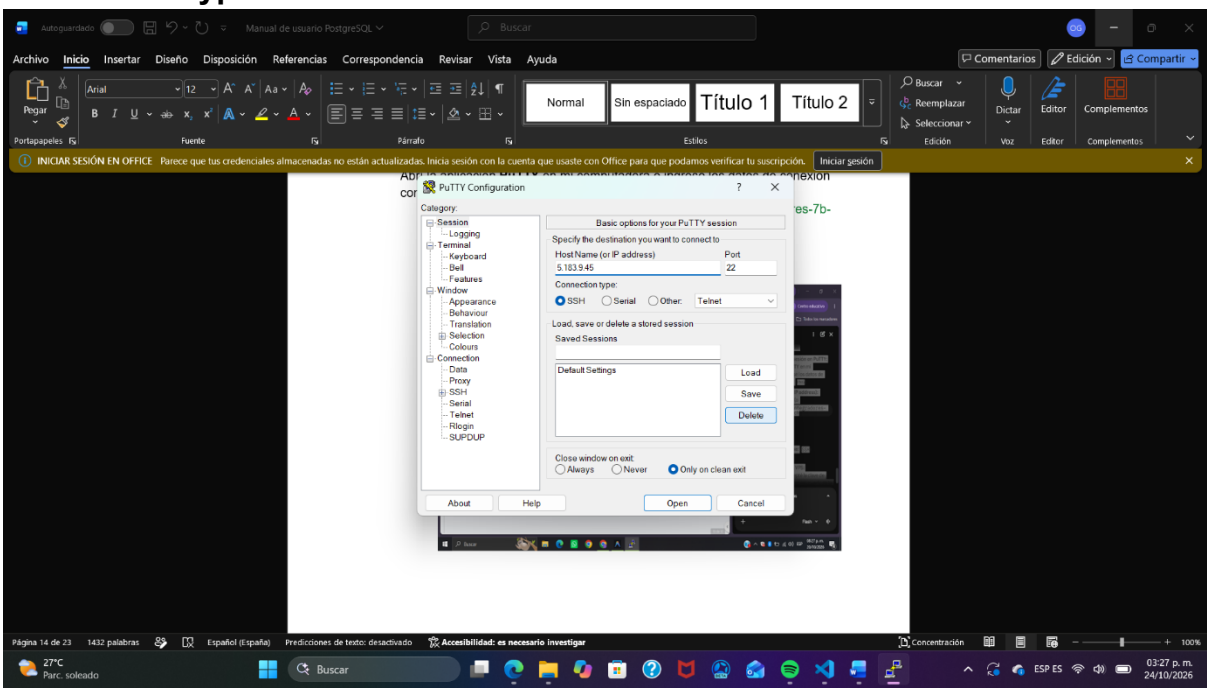
Los comandos son los siguientes:



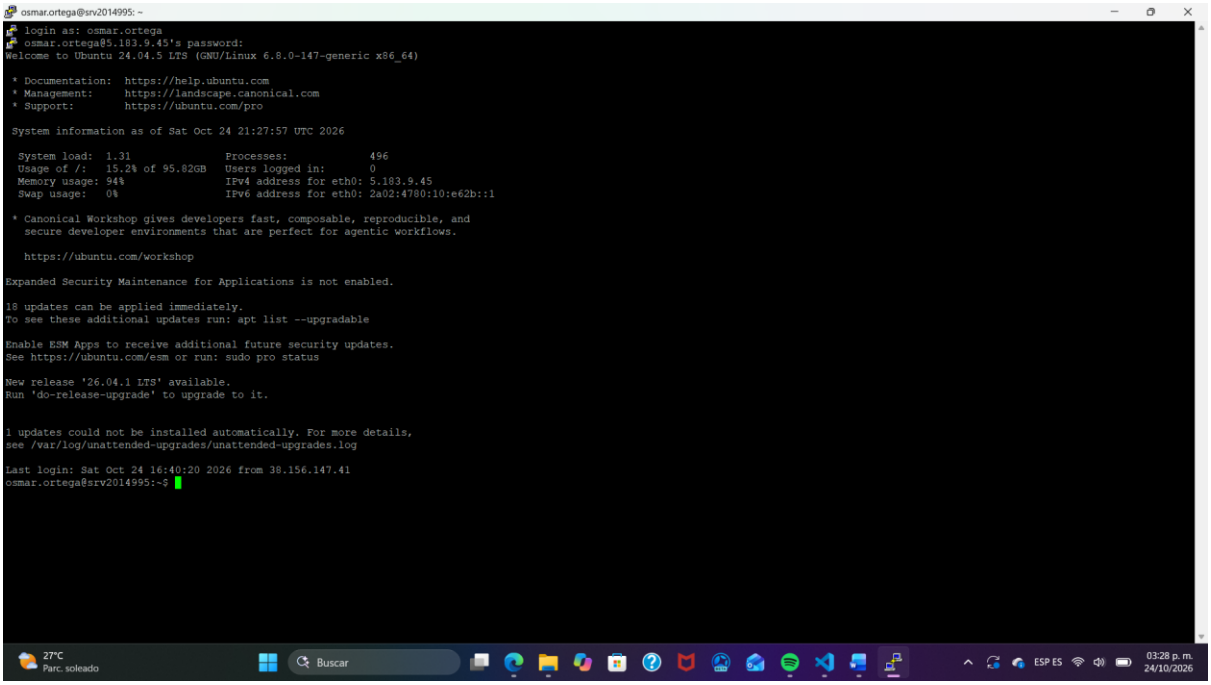
6. Conexión a la VPS con PuTTY y Clonado de Repositorio Privado

Para conectarme al servidor remoto y descargar el proyecto que se encuentra en mi repositorio privado de GitHub:

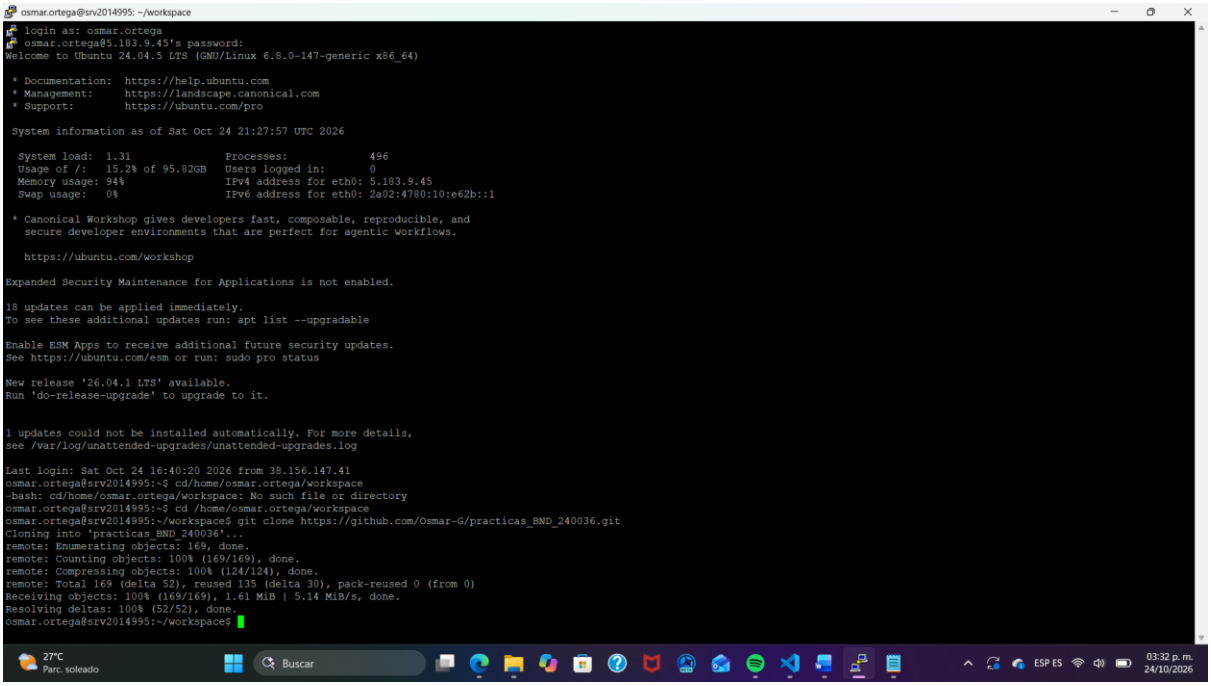
1. **Configuración de la sesión en PuTTY:**
   Abrí la aplicación **PuTTY** en mi computadora e ingresé los datos de conexión con la VPS:
   - **Host Name (or IP address):** 5.183.9.45 (o proyectos-integradores-7b-itiid.tech)
   - **Port:** 22
   - **Connection type:** SSH



2. **Inicio de sesión en la VPS:**
   Hice clic en **Open**, acepté la clave de seguridad SSH e ingresé mi usuario de la VPS (osmar.ortega) junto con mi contraseña de acceso.



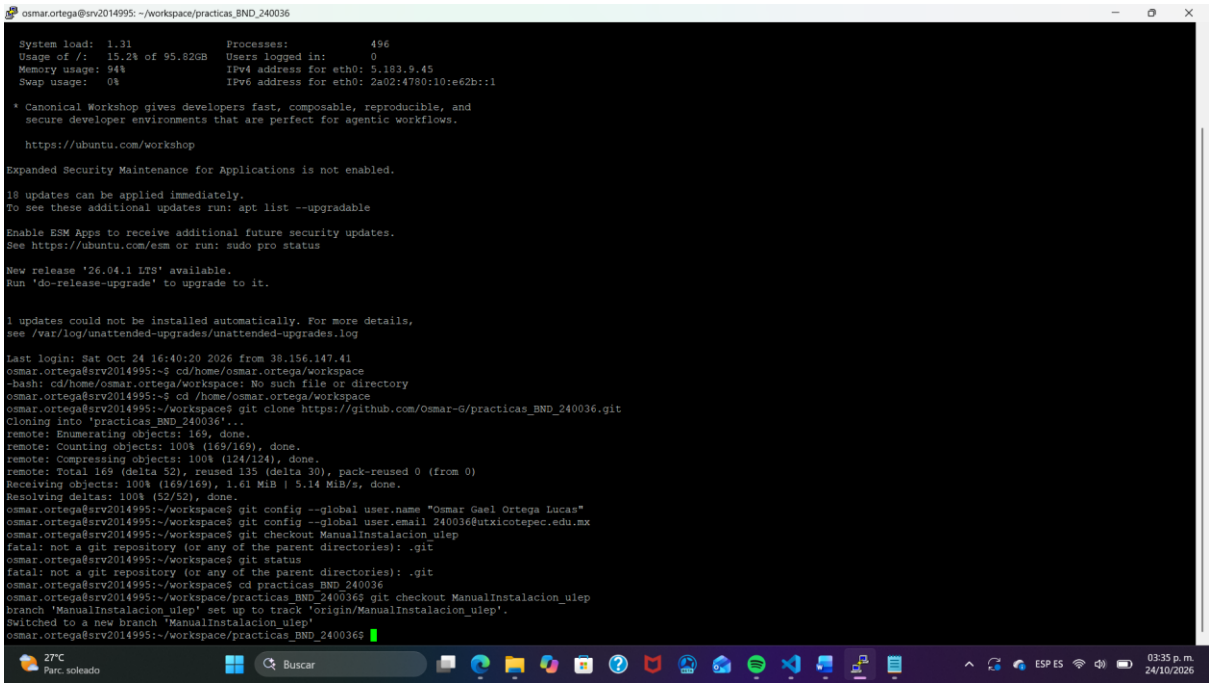
3. **Clonar el repositorio privado en la VPS:**
   Navegué a mi directorio de trabajo y ejecuté el comando git clone para descargar mi repositorio privado:



4. Autenticación con Personal Access Token:
Al tratarse de un **repositorio privado**, la terminal de la VPS me solicitó credenciales de acceso:

- **Username for 'https://github.com':** Ingresé mi usuario Osmar-G.
- **Password for 'https://Osmar_g@github.com':** Pegué mi Personal Access Token generado anteriormente (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).

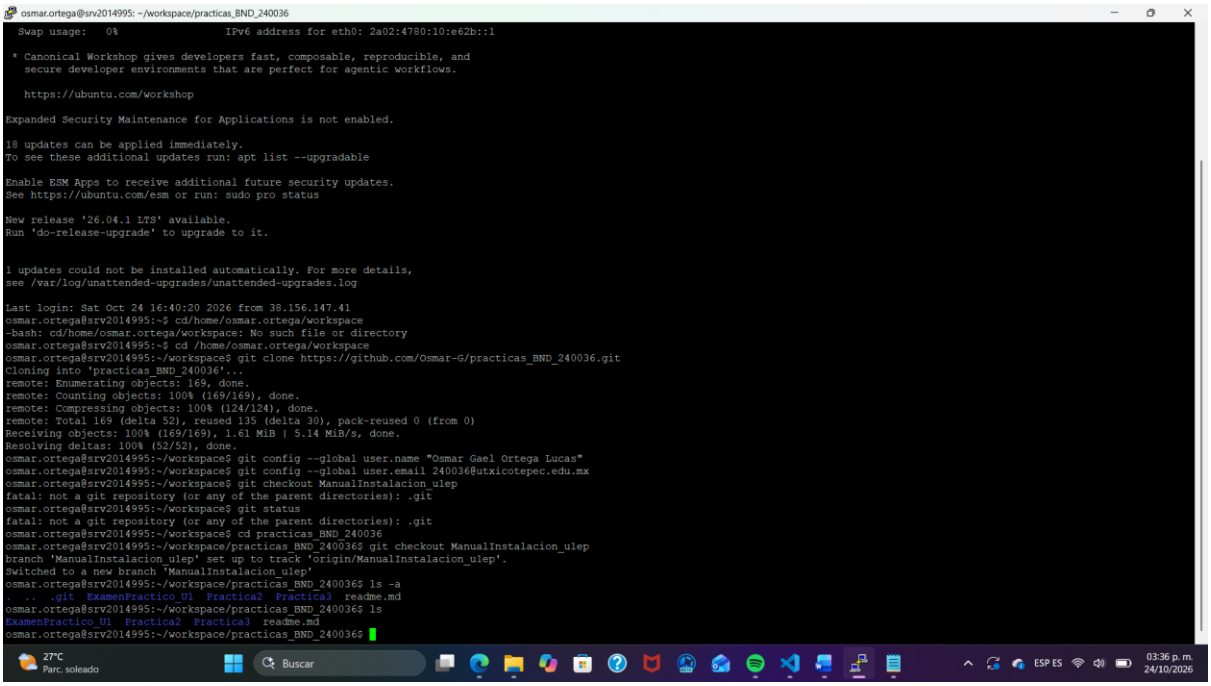
Cambiar a la rama de trabajo:



7. Cambio a mi Rama de Trabajo y Verificación de Archivos

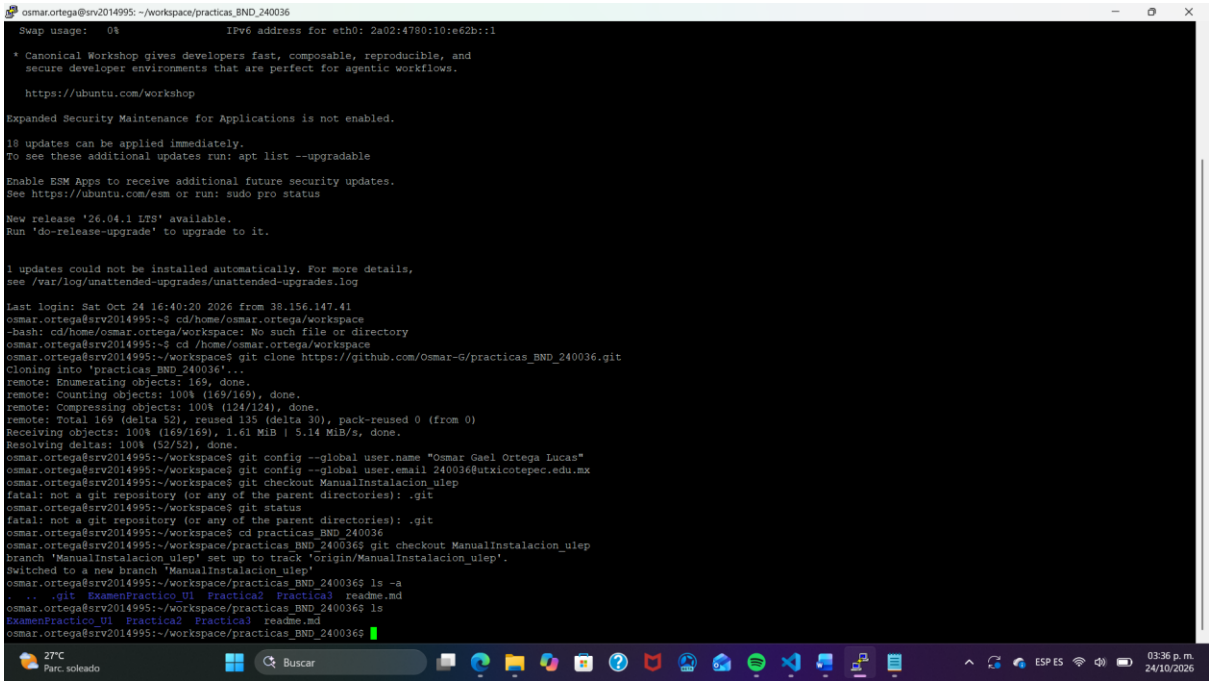
Una vez ubicado dentro del directorio de mi repositorio en la VPS:

1. **Cambiarse a la rama correspondiente:**
   Me cambié a la rama donde subí el archivo de configuración:



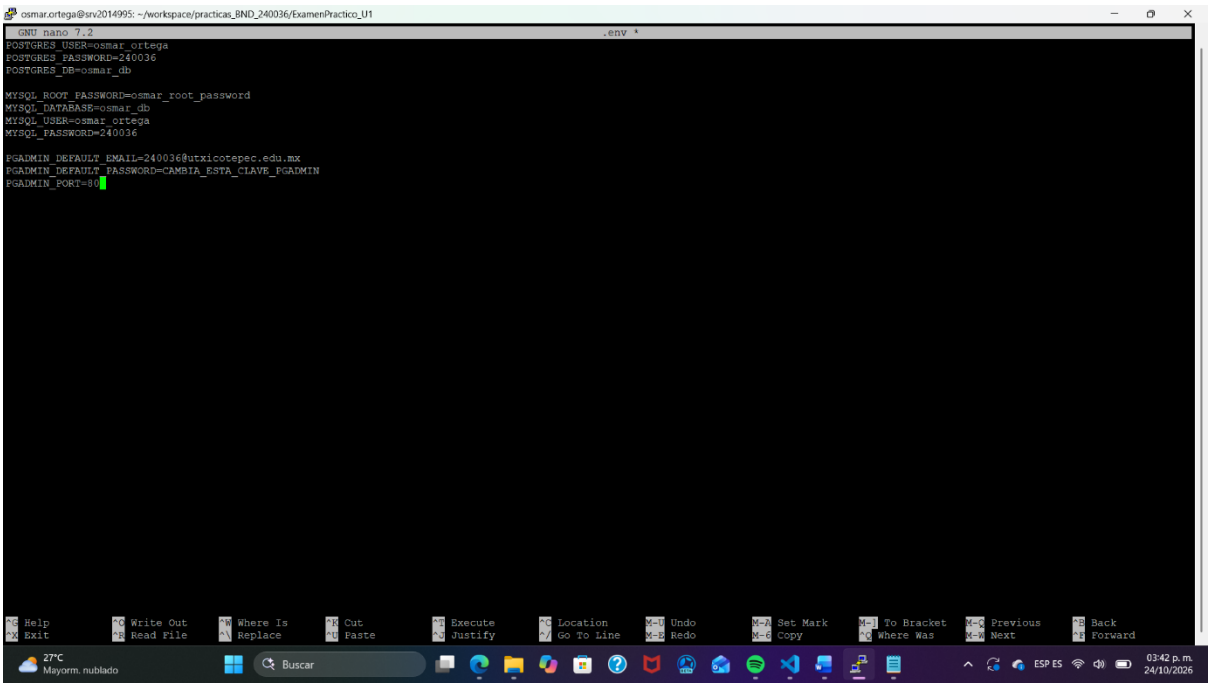
Verificar el contenido:

Comprobé que mi archivo `docker-compose.yml` se encontrara presente en el directorio con el comando ls -a:



Despliegue de Contenedores

1. ***Creacion de un archivo .env en la carpeta del proyecto:***
   En la terminal escribe nano .env y click en ENTER



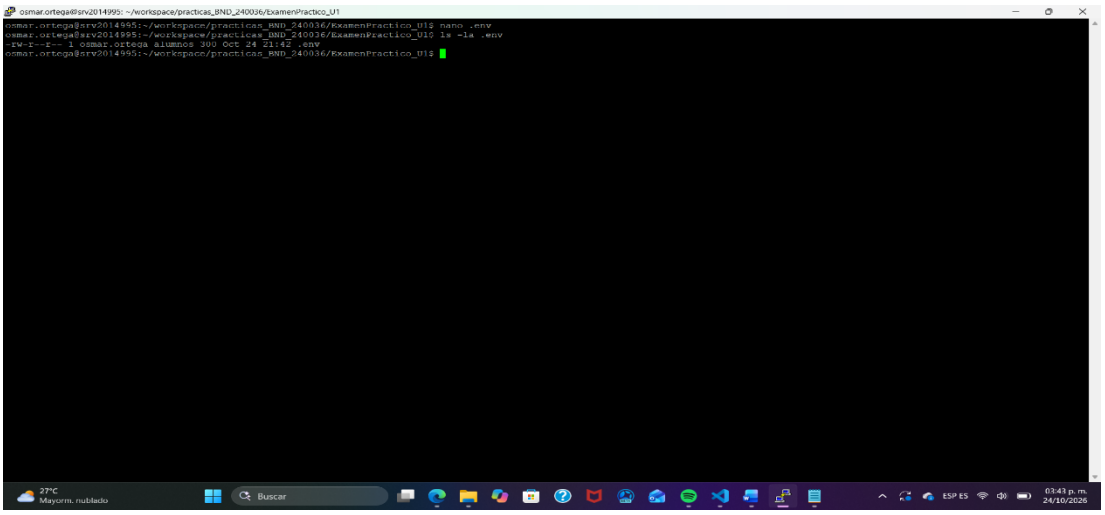
Aquí configuras el nombre y contraseñas que quieres que tenga tus servicios de SQL, POSTGRES Y PGADMIN

CTRL-S para guardar

ENTER – para continuar

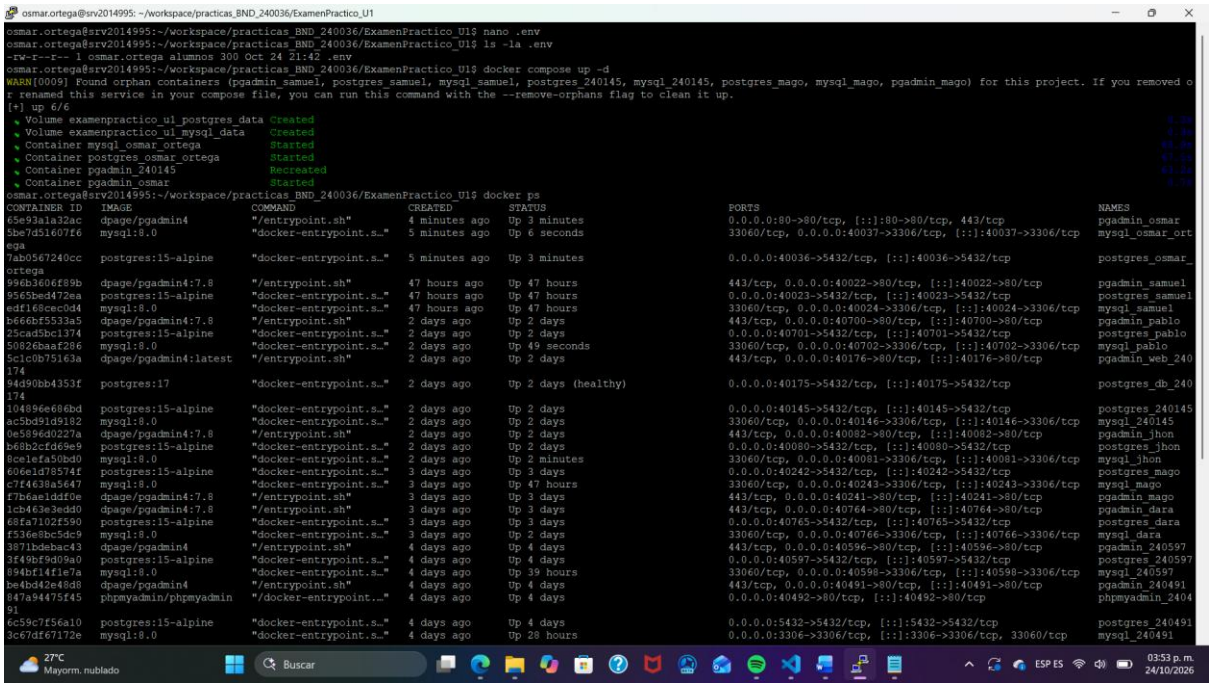
CTRL-X para salir

VERIFICAS QUE SE CREO CON el comando ls -la .env



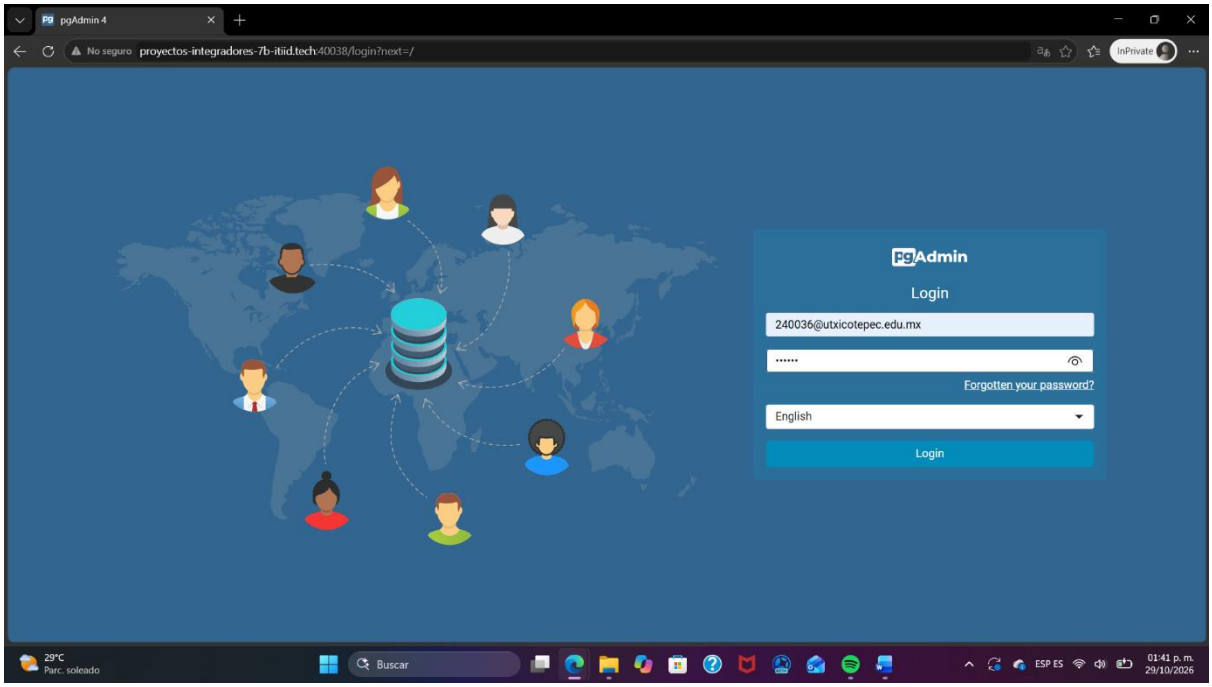
Levantar los servicios con Docker Compose:

Una vez revisado que se creo el .env escribimos docker compose up -d



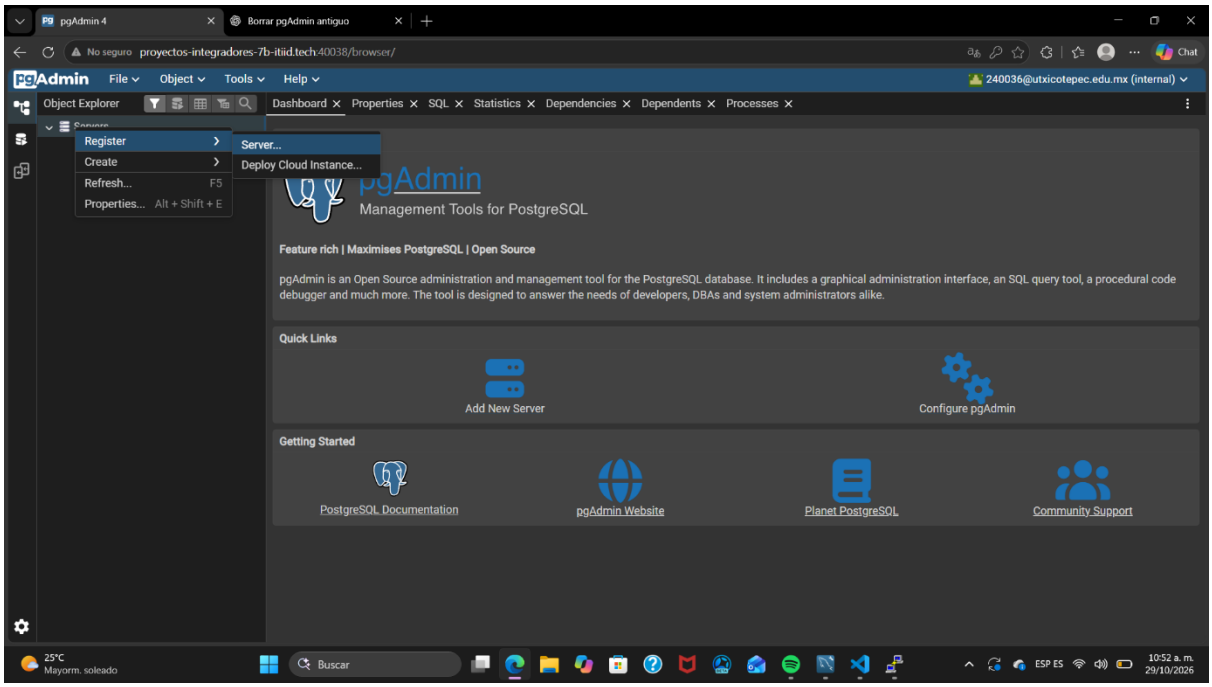
9. Inicio de Sesión y Registro de Base de Datos en pgAdmin

1. ***Acceso web a pgAdmin:***
   Desde el navegador de mi computadora, ingresé a la URL pública:
   http://proyectos-integradores-7b-itiid.tech:40038

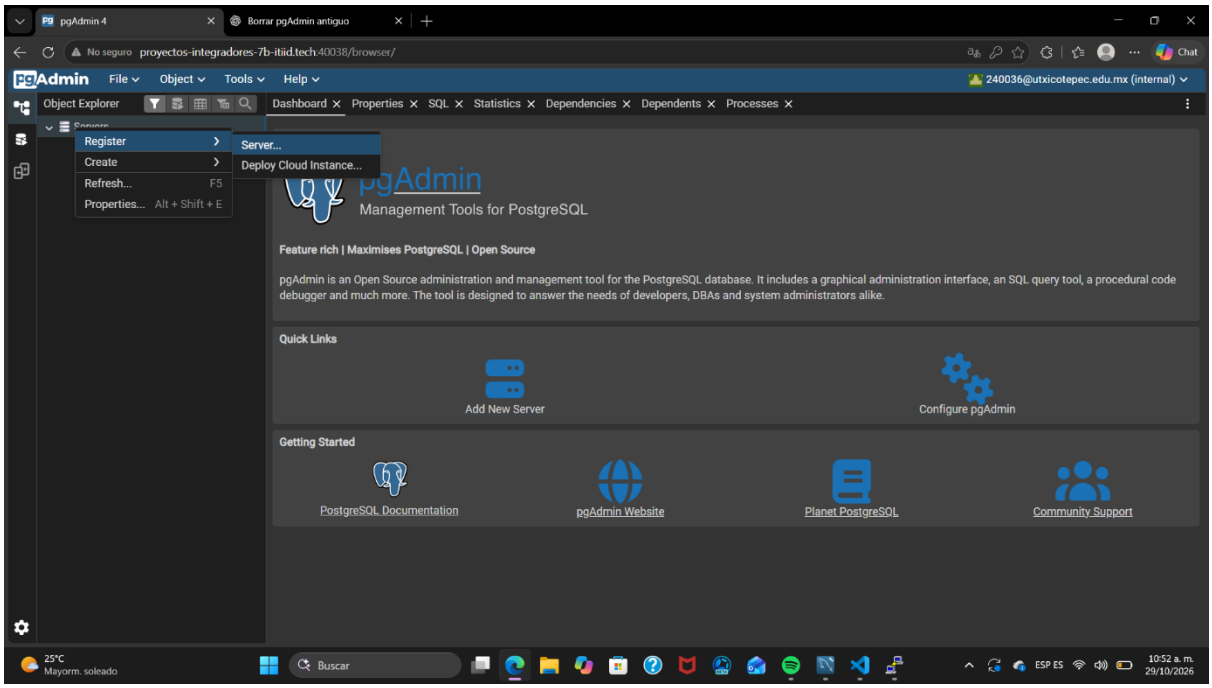


2. *Ingresamos las contraseñas*

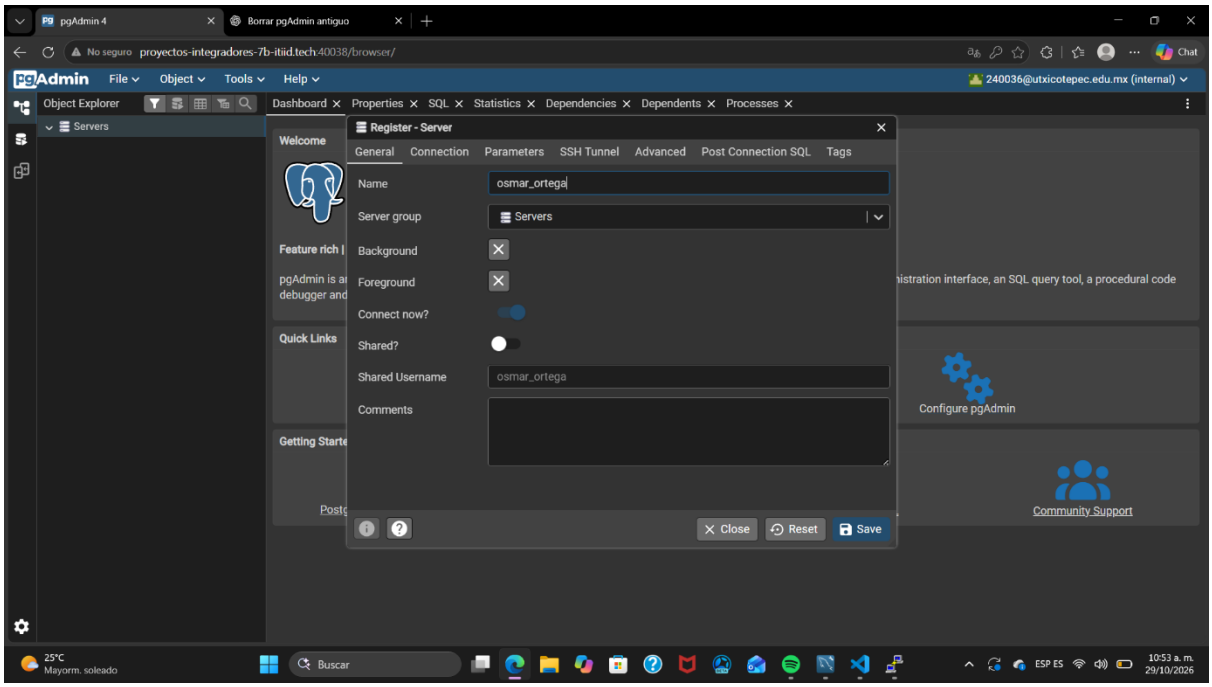
   Nos va aparecer como esto

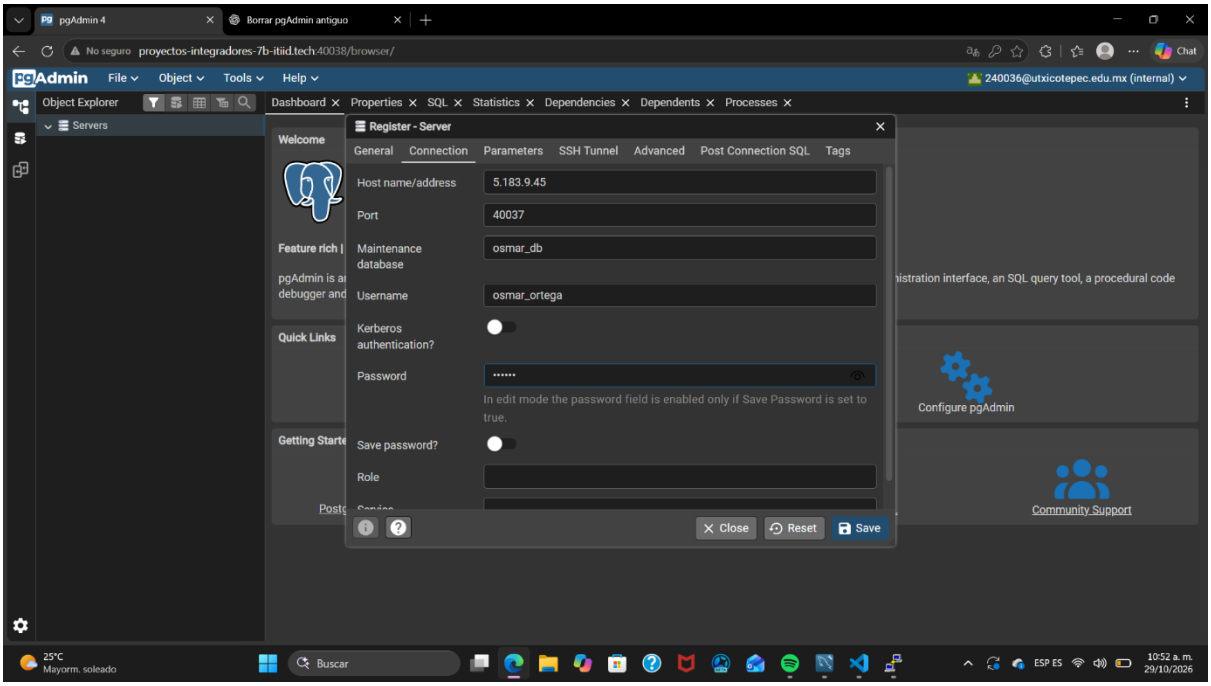


3. *Despues damos click DERECHO en **SERVERS** click en Register y por ultimo click en Servers :*

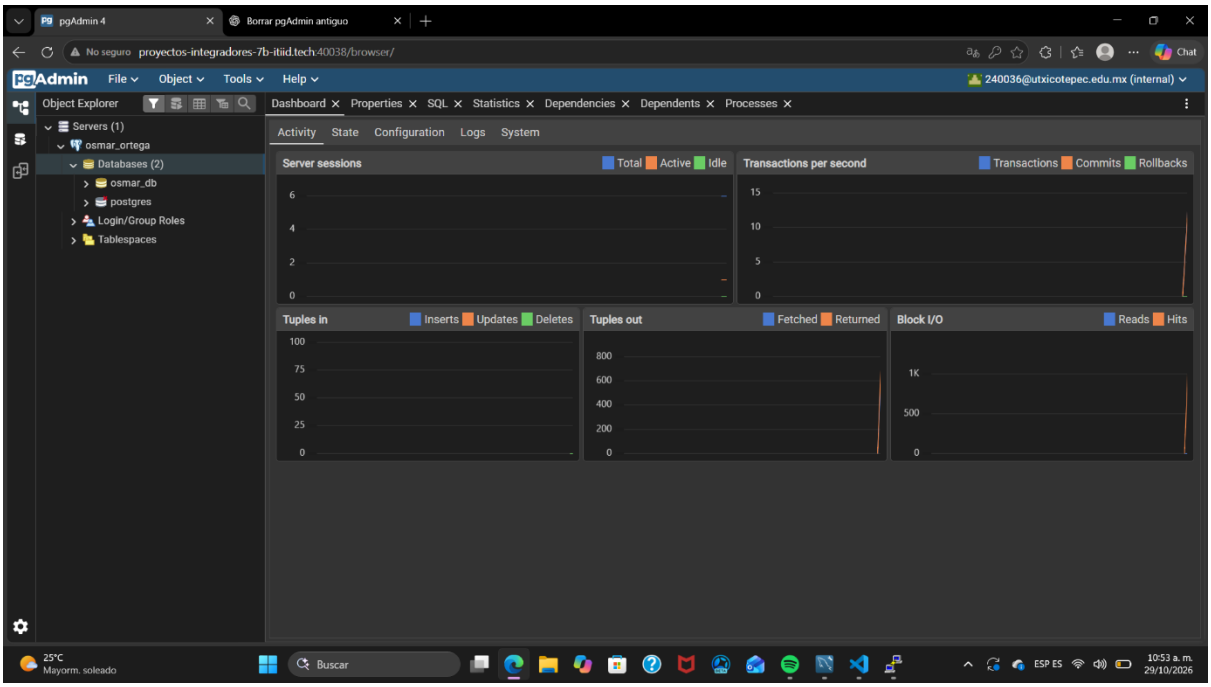


4. *Configuramos el servidor con el nombre de nuestro usuario,la ip en donde se encuentra nuestro Docker,el nombre de nuestra base de datos y la contraseña que le asignamos.*



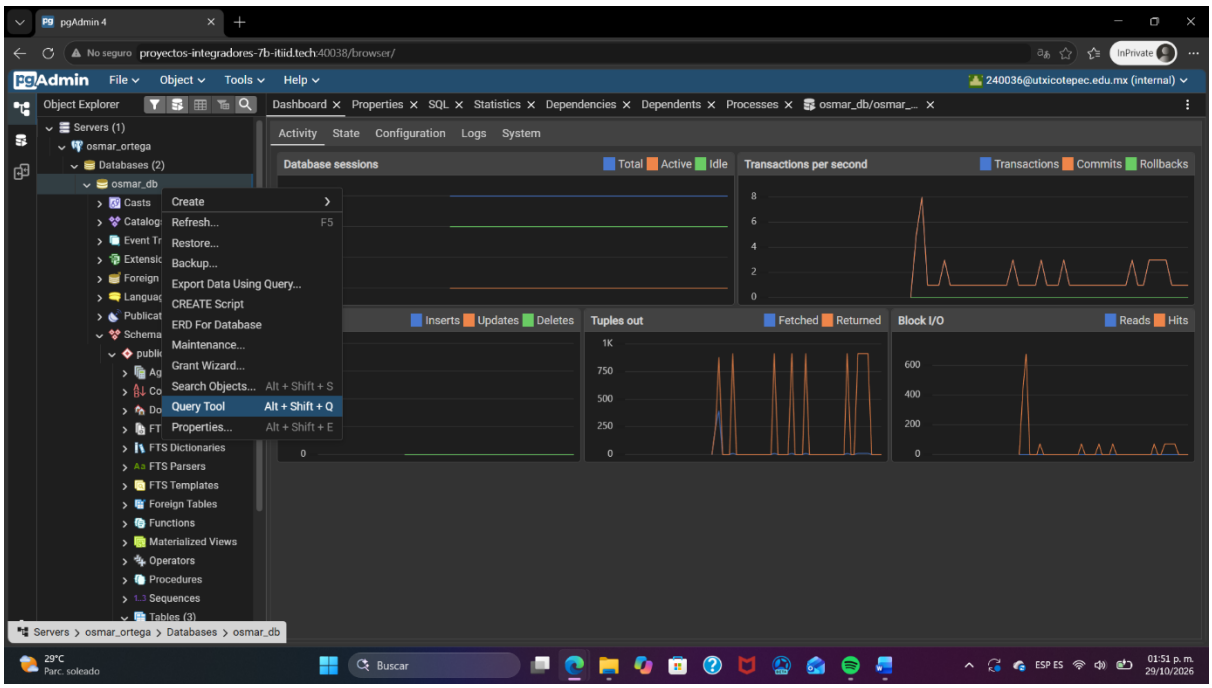


5. *Click en SAVE y nos muestra nuestras base de datos*

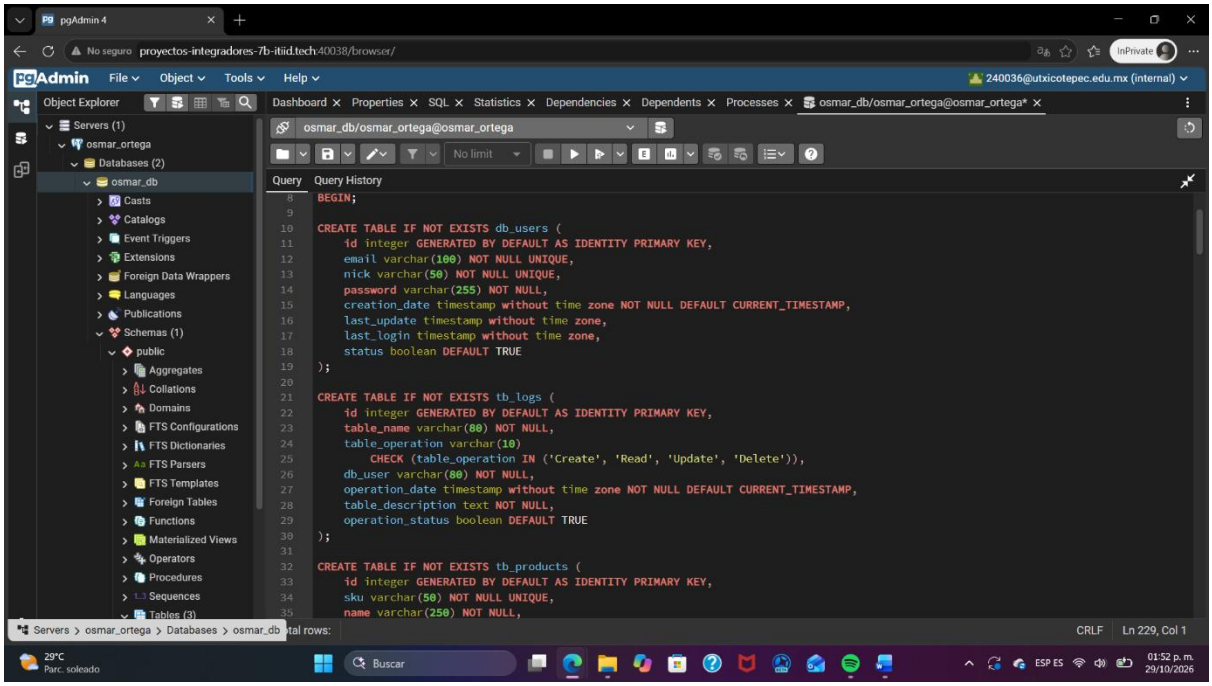


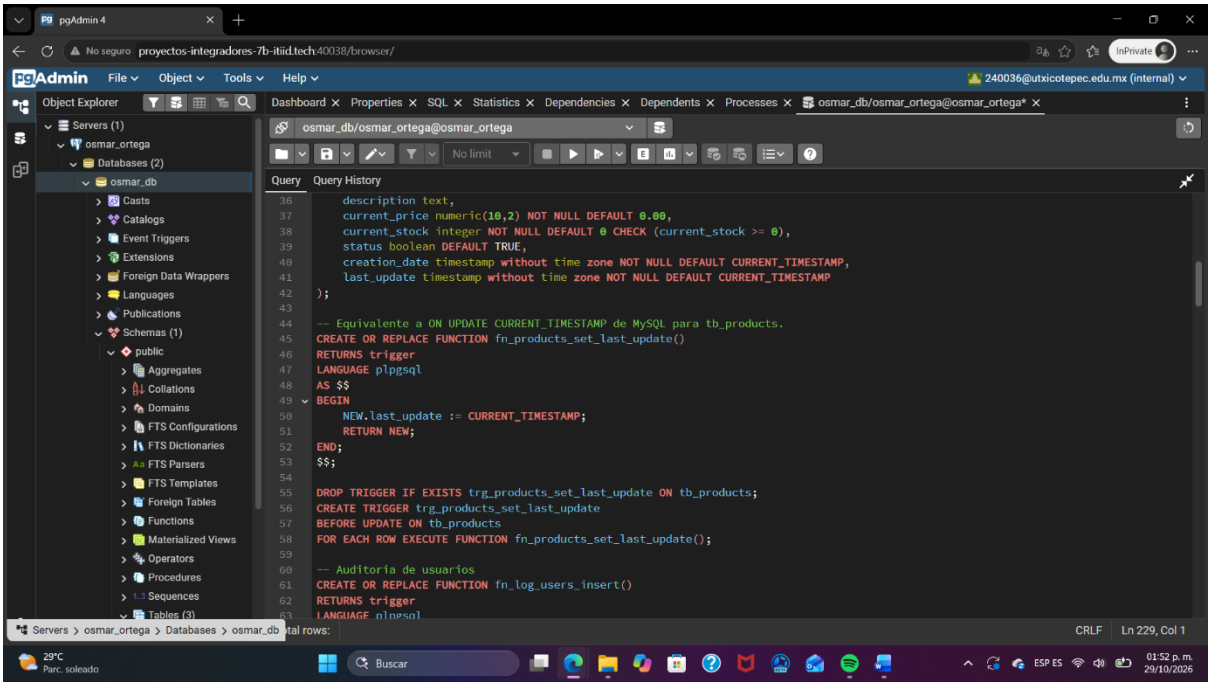
AHORA PASAREMOS A LA CREACION DE LAS TABLAS,VISTAS DE TRAZABILIDAD Y TRIGGERS

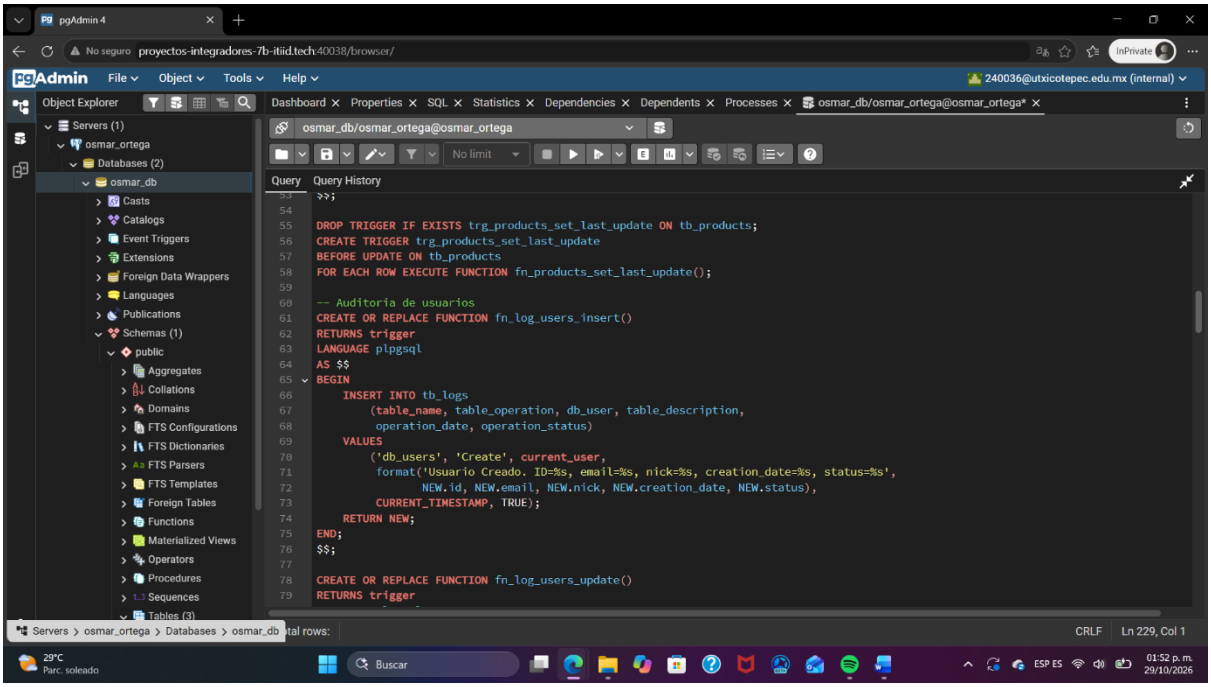
6. *CLICK DERECHO sobre la base que es osmar_db y elegimos la opción de Query Tool*



Codigo de Creacion de Tablas,Triggers



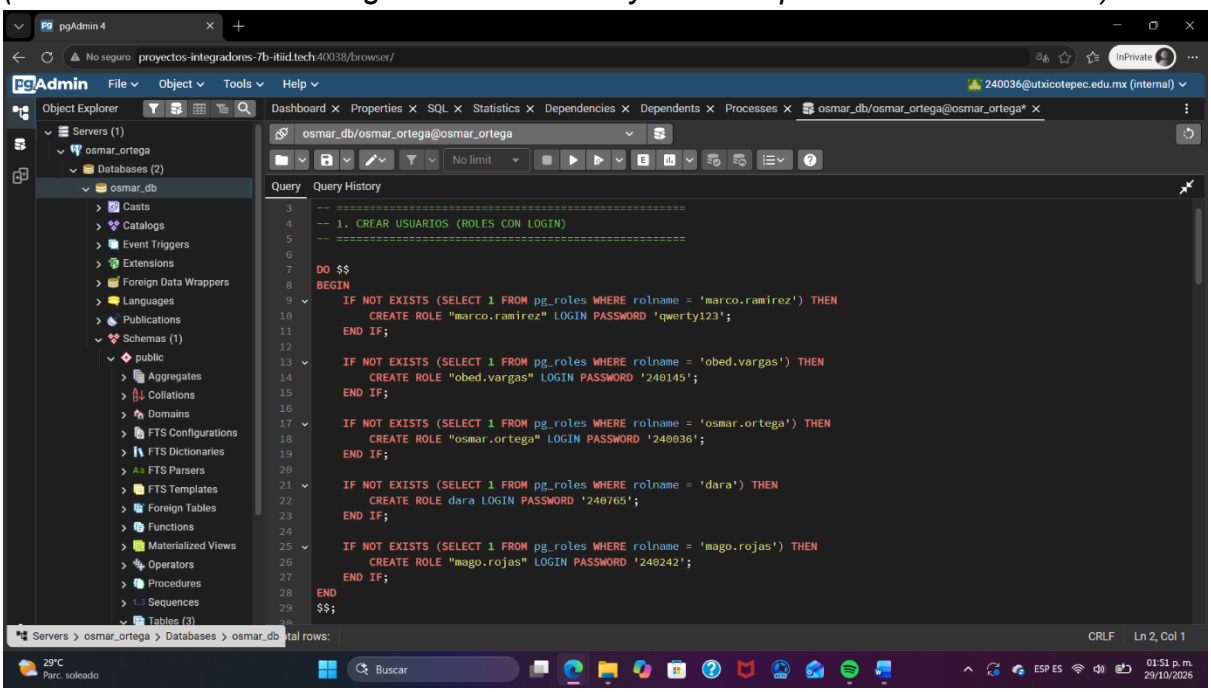


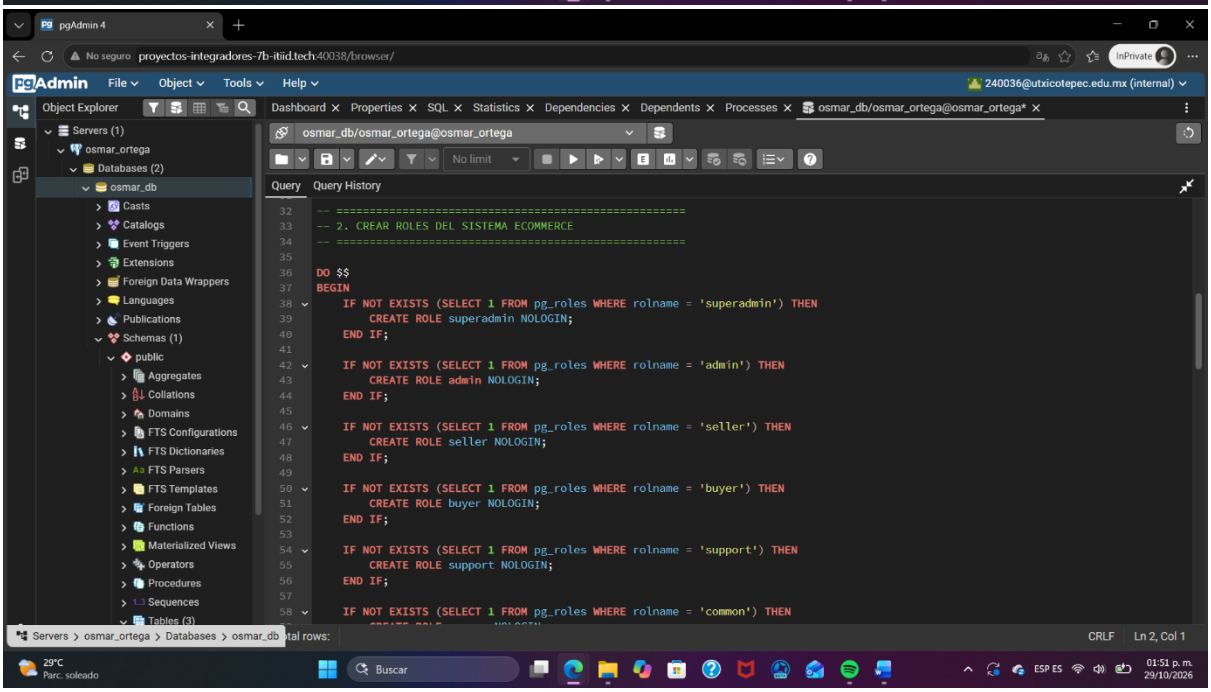


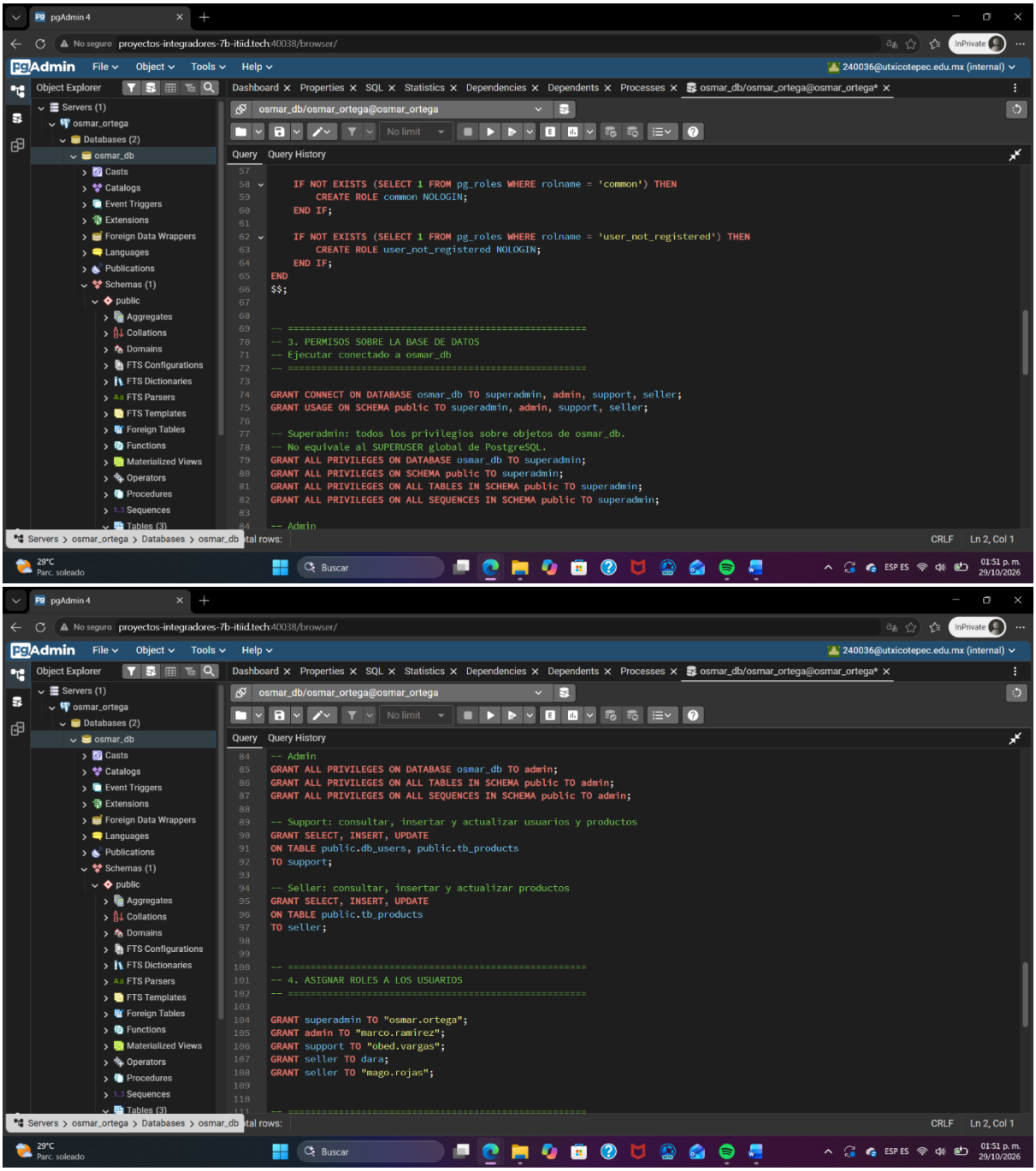
Para compilar el código damos click F5 en el teclado o el botón de play.

7. *Ahora pasaremos a la creación de roles y permisos*

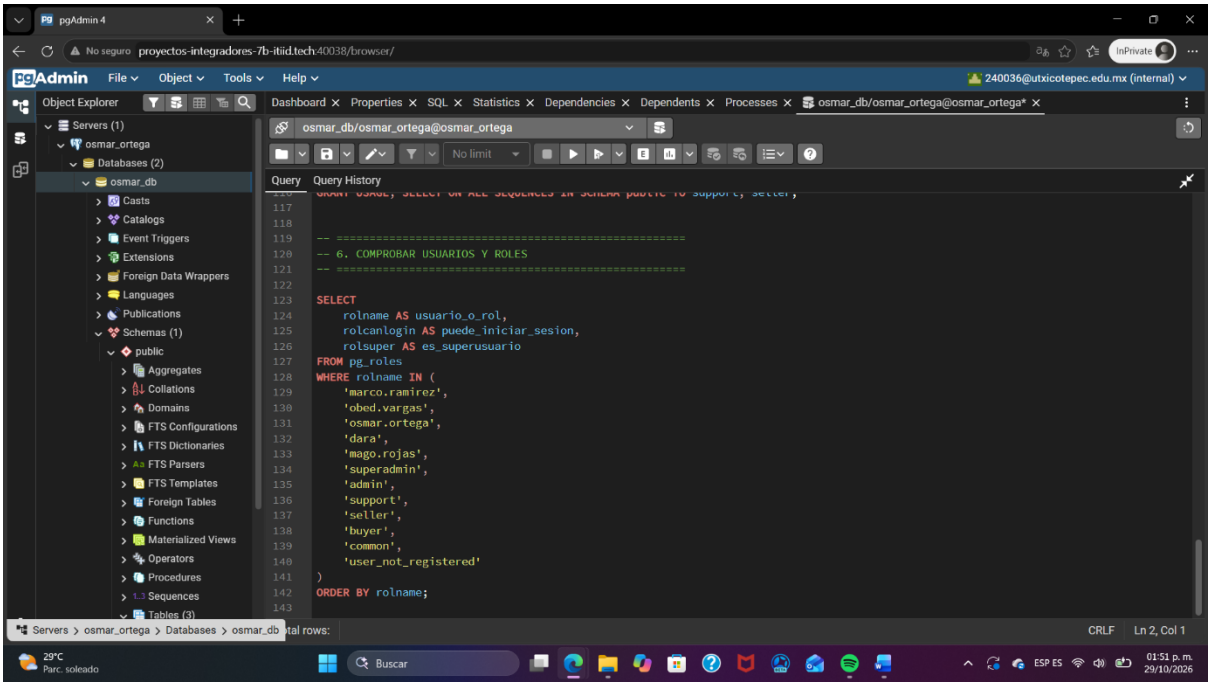
(Se borra el código existente y se pone este nuevo)





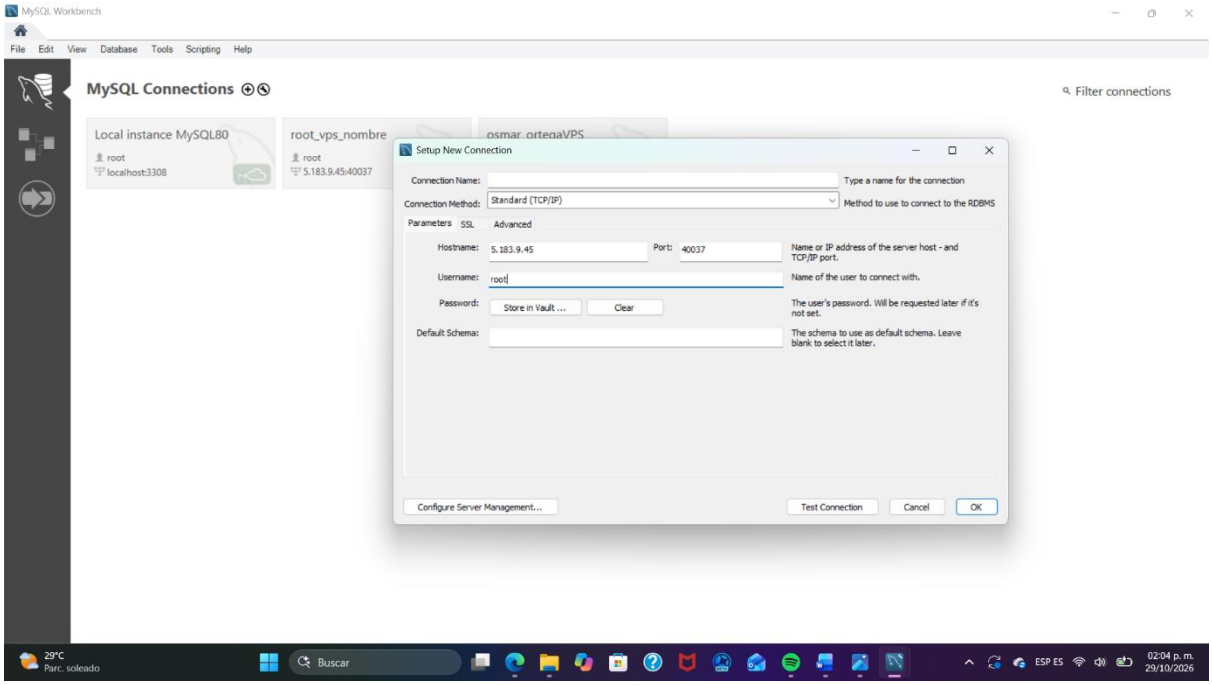


Y por ultimo esta consulta para verificar que fueron creados nuestros roles y usuarios.

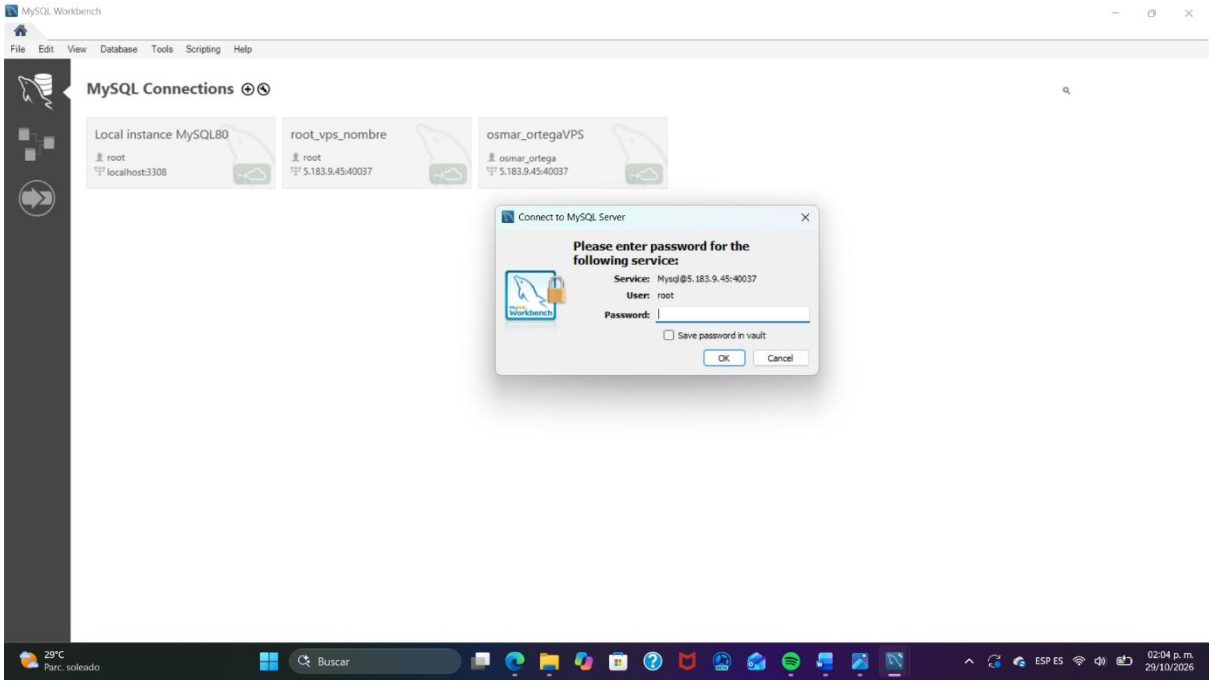


Configuracion de MySQL

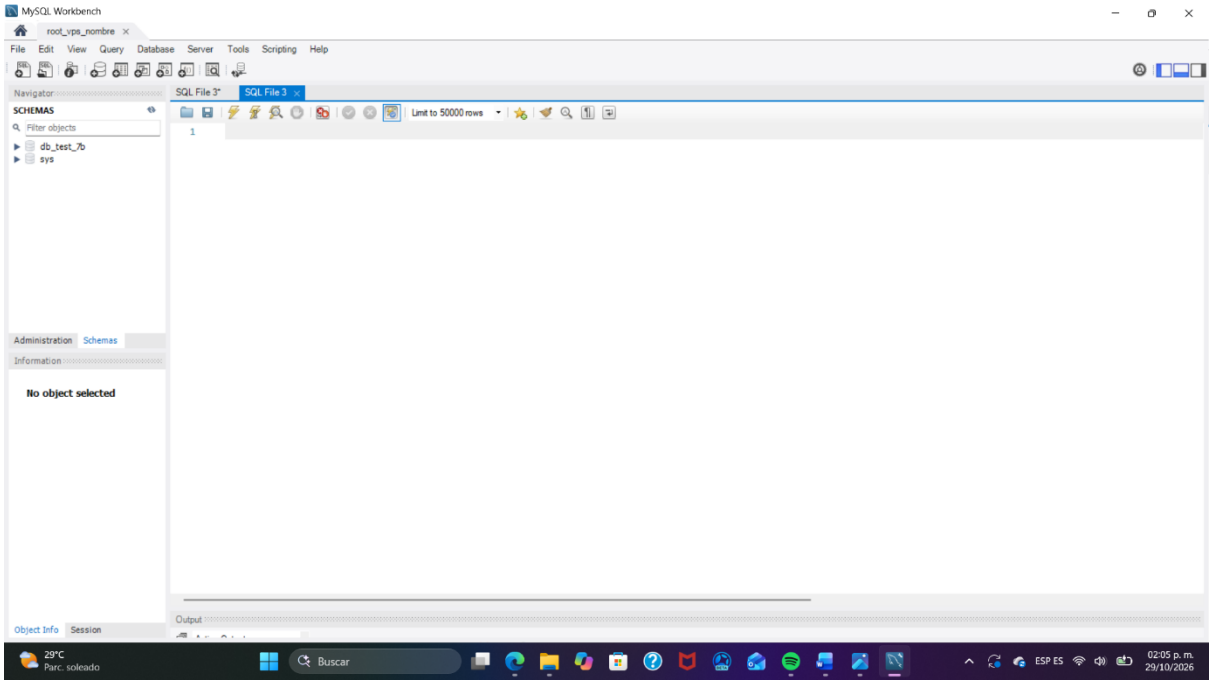
1.Abrir Workbench y en MySQL Connections dar click en el símbolo de mas
Configuramos nuestro SQL con los datos que configuramos en el DOCKER



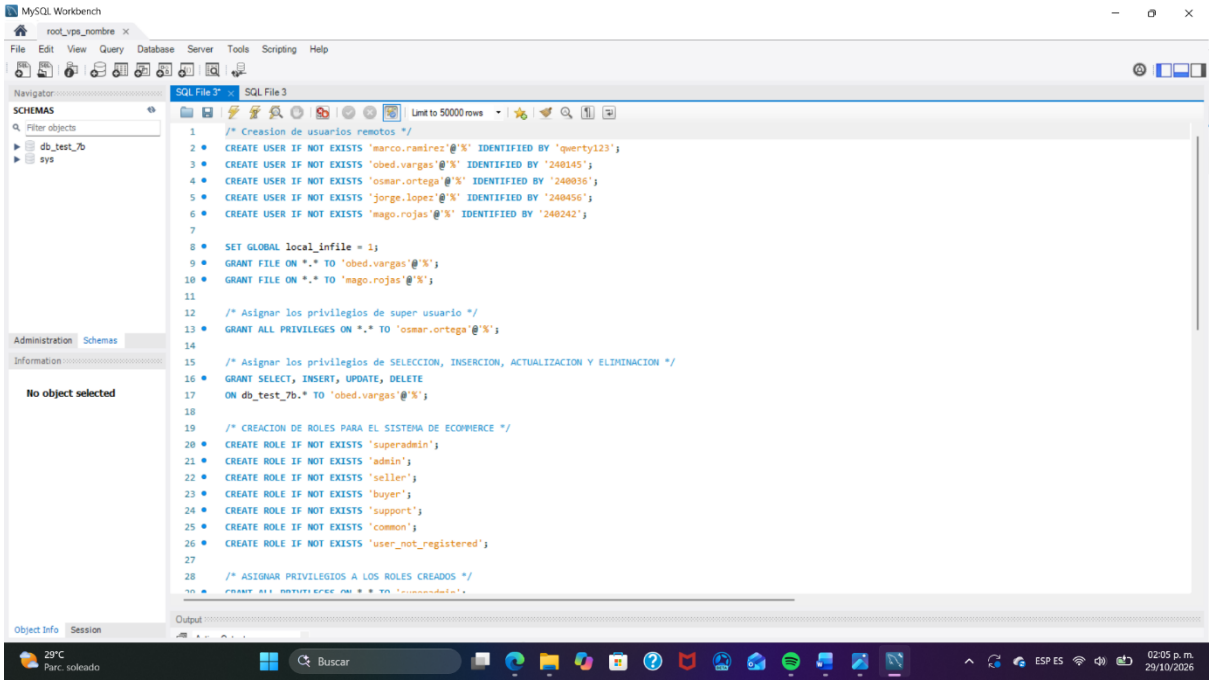
Dar click en OK
Nos va aparecer esta pantalla,tenemos que ingresar la contraseña que en este caso es mi matricula

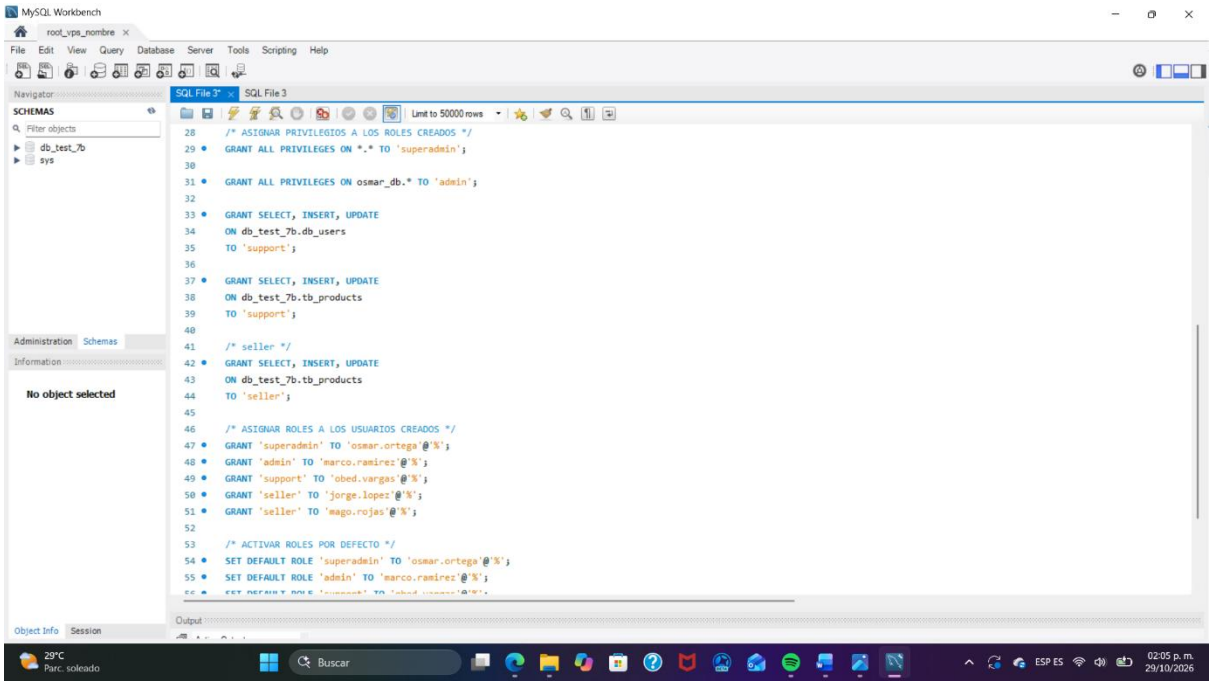


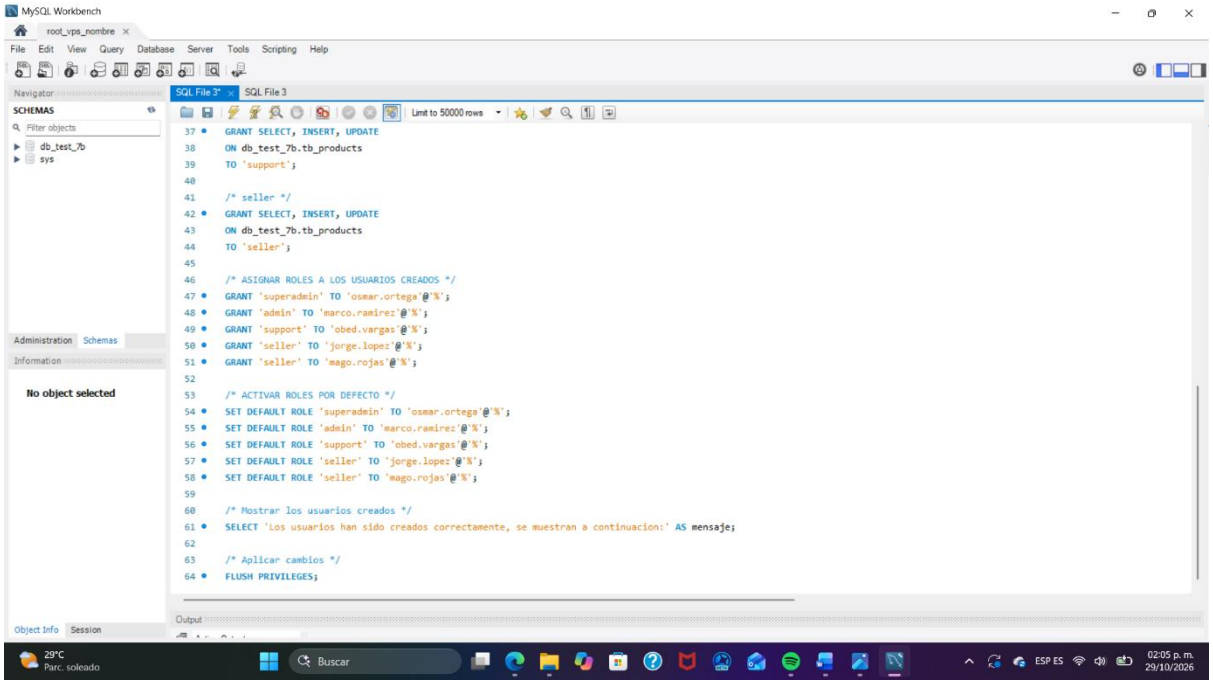
Si toda esta bien nos va a mostrar la siguiente pantalla



Una ves dentro seleccionamos la base de datos que en este caso es la de db_test_7b e ingresamos los siguientes permisos

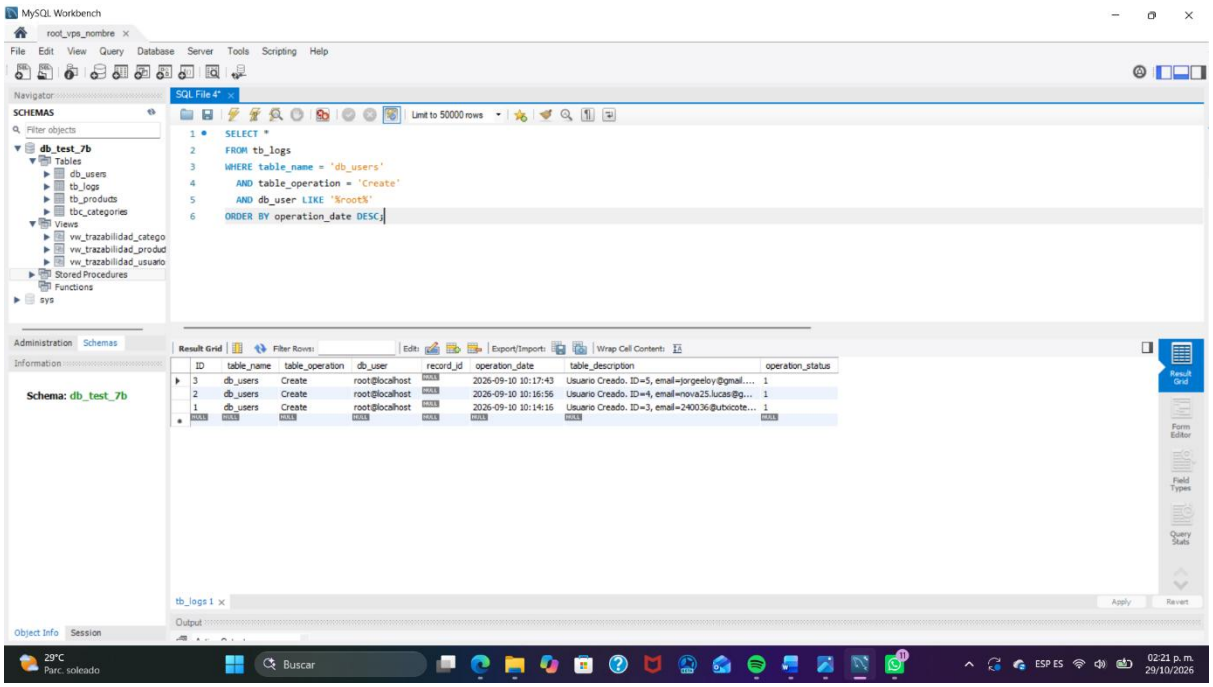

MySQL Workbench
root_vps_nombre
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
db_test_7b
sys
SQL File 3*
SQL File 3
Limit to 50000 rows
/* Creasion de usuarios remotos */
CREATE USER IF NOT EXISTS 'marco.ramirez'@'%' IDENTIFIED BY 'qwerty123';
CREATE USER IF NOT EXISTS 'obed.vargas'@'%' IDENTIFIED BY '240145';
CREATE USER IF NOT EXISTS 'osmar.ortega'@'%' IDENTIFIED BY '240036';
CREATE USER IF NOT EXISTS 'jorge.lopez'@'%' IDENTIFIED BY '240456';
CREATE USER IF NOT EXISTS 'mago.rojas'@'%' IDENTIFIED BY '240242';
SET GLOBAL local_infile = 1;
GRANT FILE ON *.* TO 'obed.vargas'@'%';
GRANT FILE ON *.* TO 'mago.rojas'@'%';
/* Asignar los privilegios de super usuario */
GRANT ALL PRIVILEGES ON *.* TO 'osmar.ortega'@'%';
/* Asignar los privilegios de SELECCION, INSERCION, ACTUALIZACION Y ELIMINACION */
GRANT SELECT, INSERT, UPDATE, DELETE
ON db_test_7b.* TO 'obed.vargas'@'%';
/* CREACION DE ROLES PARA EL SISTEMA DE ECOMMERCE */
CREATE ROLE IF NOT EXISTS 'superadmin';
CREATE ROLE IF NOT EXISTS 'admin';
CREATE ROLE IF NOT EXISTS 'seller';
CREATE ROLE IF NOT EXISTS 'buyer';
CREATE ROLE IF NOT EXISTS 'support';
CREATE ROLE IF NOT EXISTS 'common';
CREATE ROLE IF NOT EXISTS 'user_not_registered';
/* ASIGNAR PRIVILEGIOS A LOS ROLES CREADOS */
Administration Schemas
Information
No object selected
Object Info Session
Output
29°C Parc. soleado
Buscar
ESP ES
02:05 p. m. 29/10/2026


MySQL Workbench
root_vps_nombre
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
db_test_7b
sys
SQL File 3*
SQL File 3
Limit to 50000 rows
/* ASIGNAR PRIVILEGIOS A LOS ROLES CREADOS */
GRANT ALL PRIVILEGES ON *.* TO 'superadmin';
GRANT ALL PRIVILEGES ON osmar_db.* TO 'admin';
GRANT SELECT, INSERT, UPDATE
ON db_test_7b.db_users
TO 'support';
GRANT SELECT, INSERT, UPDATE
ON db_test_7b.tb_products
TO 'support';
/* seller */
GRANT SELECT, INSERT, UPDATE
ON db_test_7b.tb_products
TO 'seller';
/* ASIGNAR ROLES A LOS USUARIOS CREADOS */
GRANT 'superadmin' TO 'osmar.ortega'@'%';
GRANT 'admin' TO 'marco.ramirez'@'%';
GRANT 'support' TO 'obed.vargas'@'%';
GRANT 'seller' TO 'jorge.lopez'@'%';
GRANT 'seller' TO 'mago.rojas'@'%';
/* ACTIVAR ROLES POR DEFECTO */
SET DEFAULT ROLE 'superadmin' TO 'osmar.ortega'@'%';
SET DEFAULT ROLE 'admin' TO 'marco.ramirez'@'%';
Administration Schemas
Information
No object selected
Object Info Session
Output
29°C Parc. soleado
Buscar
ESP ES
02:05 p. m. 29/10/2026

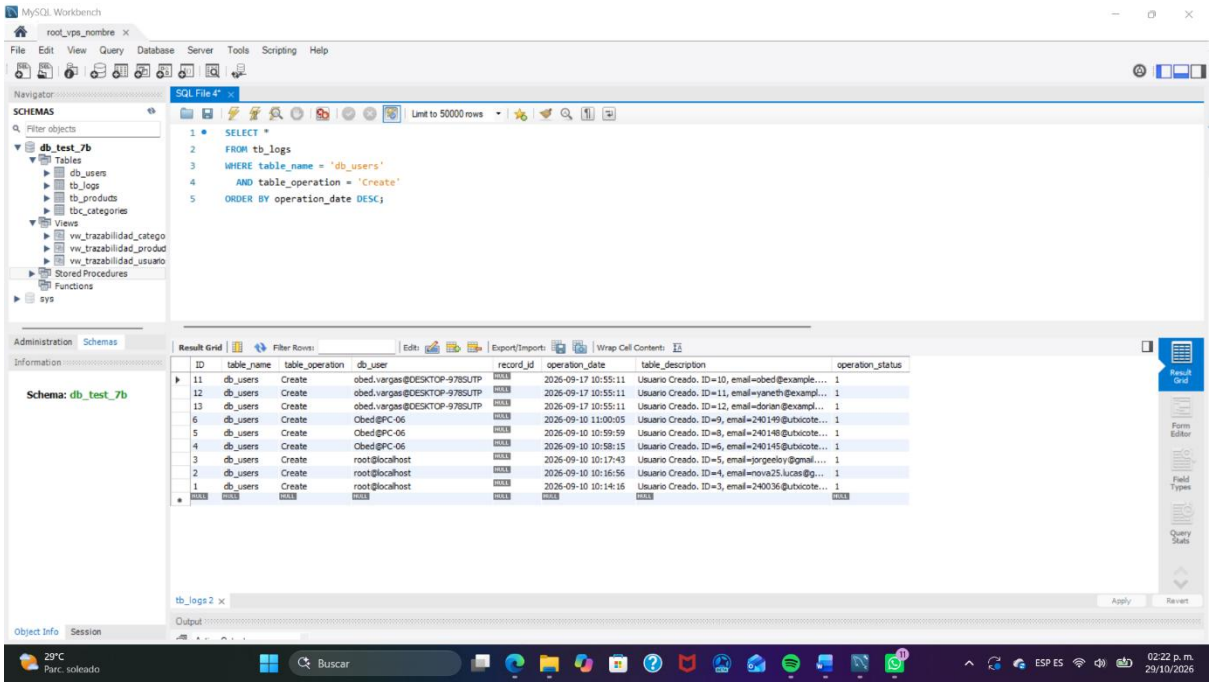

MySQL Workbench
root_vps_nombre
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
db_test_7b
sys
Administration Schemas
Information
No object selected
Object Info Session
SQL File 3* SQL File 3
Limit to 50000 rows
37 GRANT SELECT, INSERT, UPDATE
38 ON db_test_7b.tb_products
39 TO 'support';
40
41 /* seller */
42 GRANT SELECT, INSERT, UPDATE
43 ON db_test_7b.tb_products
44 TO 'seller';
45
46 /* ASIGNAR ROLES A LOS USUARIOS CREADOS */
47 GRANT 'superadmin' TO 'osmar.ortega'@'%';
48 GRANT 'admin' TO 'marco.ramirez'@'%';
49 GRANT 'support' TO 'obed.vargas'@'%';
50 GRANT 'seller' TO 'jorge.lopez'@'%';
51 GRANT 'seller' TO 'mago.rojas'@'%';
52
53 /* ACTIVAR ROLES POR DEFECTO */
54 SET DEFAULT ROLE 'superadmin' TO 'osmar.ortega'@'%';
55 SET DEFAULT ROLE 'admin' TO 'marco.ramirez'@'%';
56 SET DEFAULT ROLE 'support' TO 'obed.vargas'@'%';
57 SET DEFAULT ROLE 'seller' TO 'jorge.lopez'@'%';
58 SET DEFAULT ROLE 'seller' TO 'mago.rojas'@'%';
59
60 /* Mostrar los usuarios creados */
61 SELECT 'Los usuarios han sido creados correctamente, se muestran a continuacion:' AS mensaje;
62
63 /* Aplicar cambios */
64 FLUSH PRIVILEGES;
Output
29°C
Parc. soleado
Buscar
ESP ES
02:05 p. m.
29/10/2026

10 ejercicios en MySQL

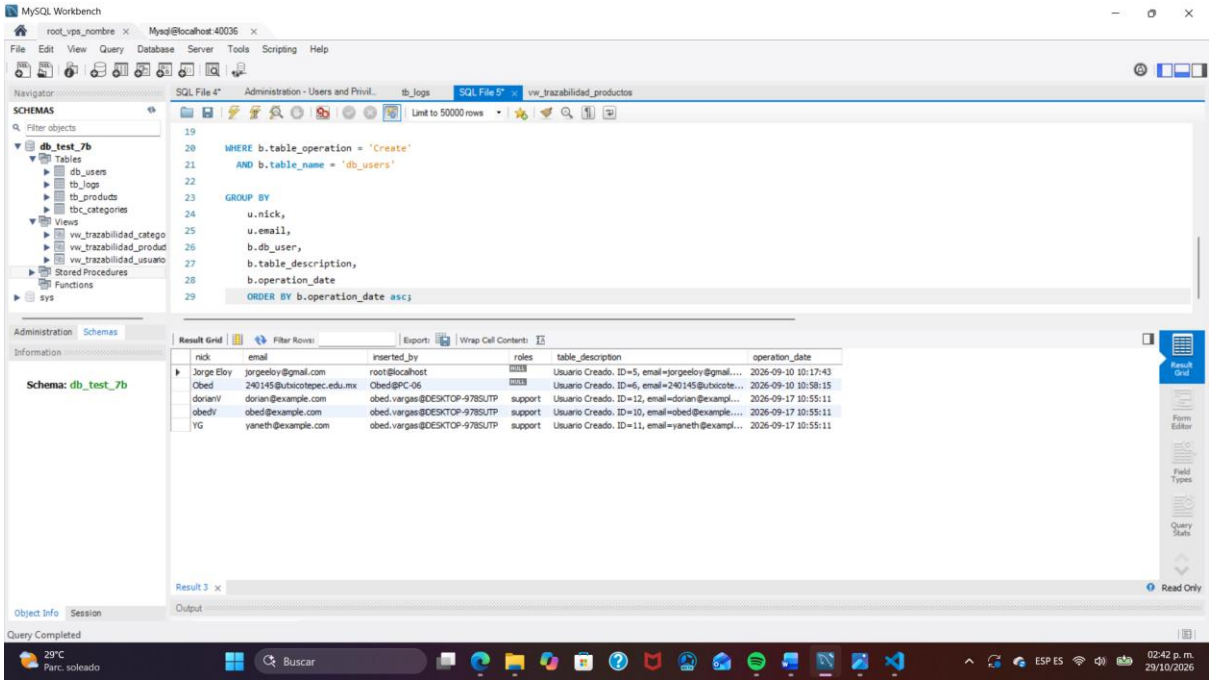
1.Usuario root inserta 3 usuarios



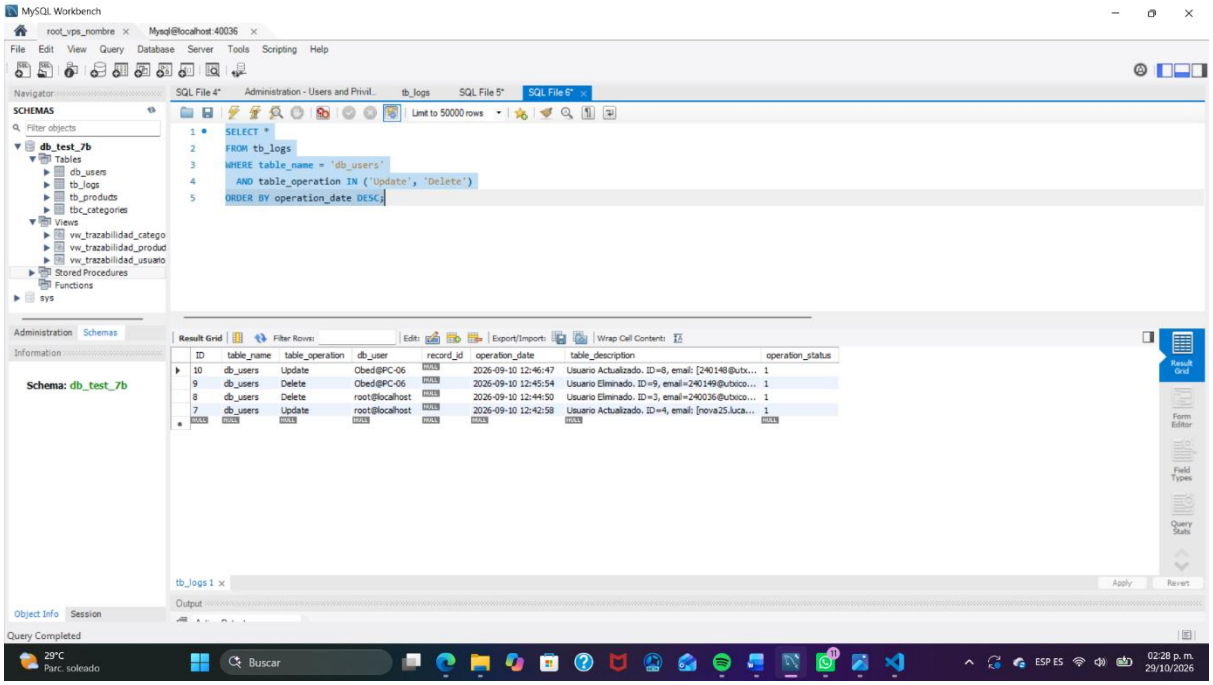
2.Usuario sin rol inserta 3 usuarios



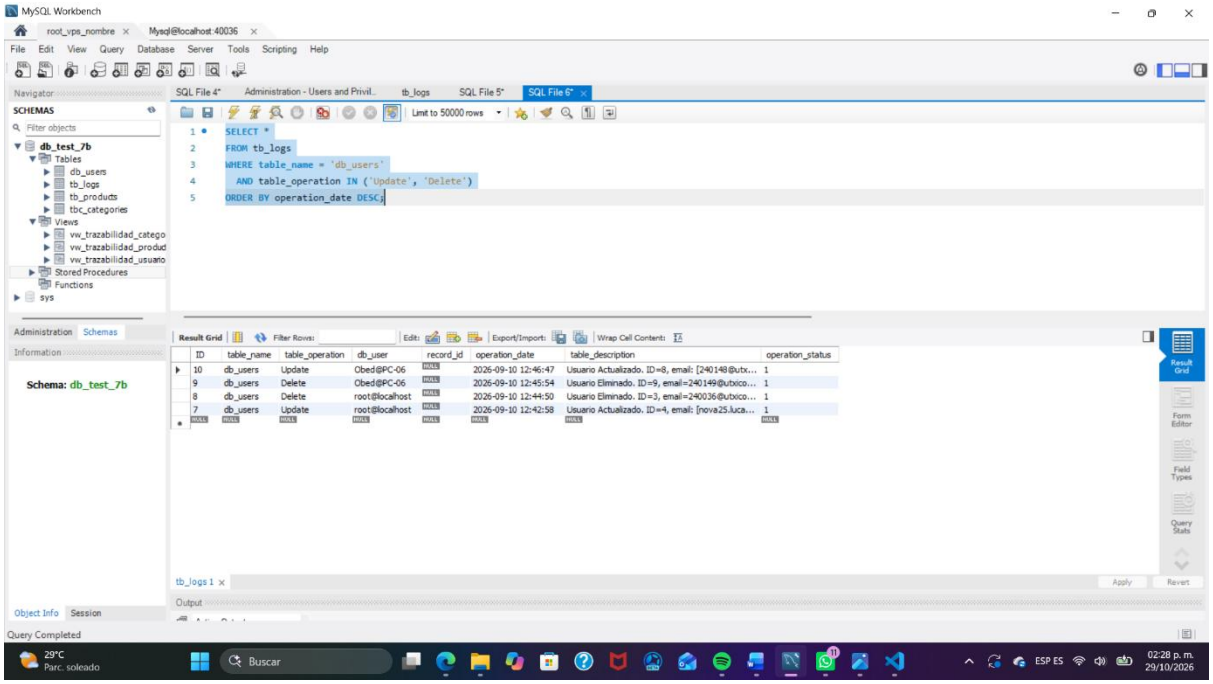
3.Usuario support inserta 3 usuarios



4.Root elimina y actualiza un usuario

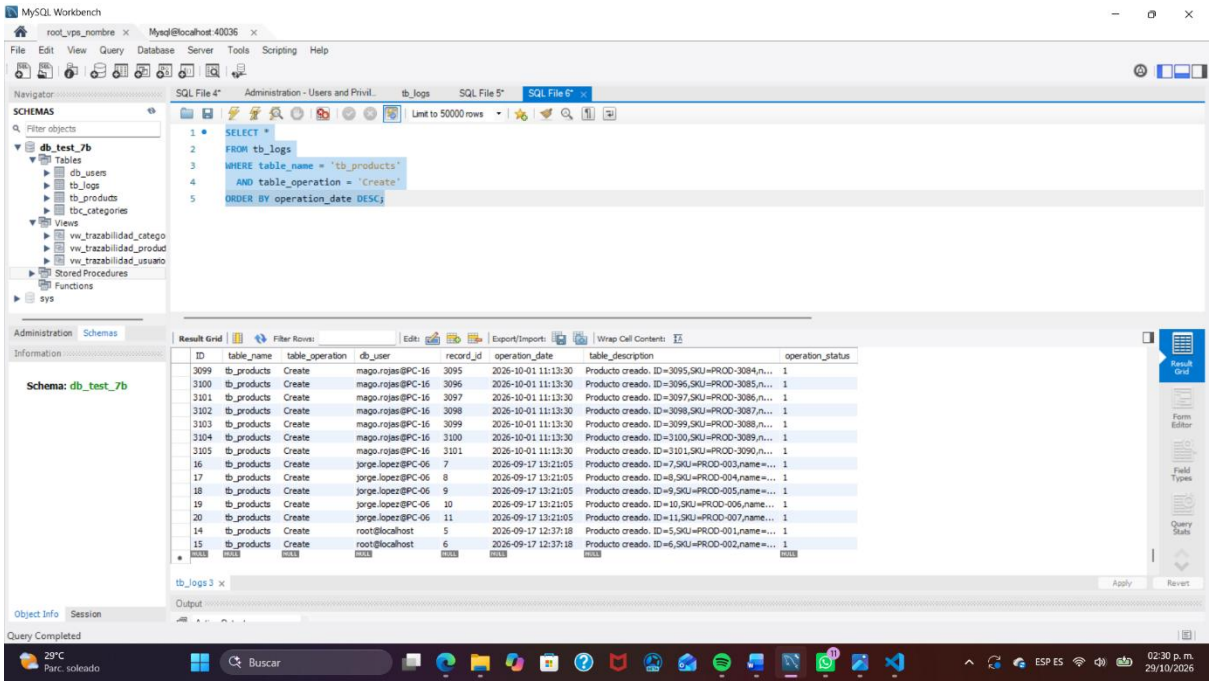


5.Usuario sin rol elimina y modifica un usuario

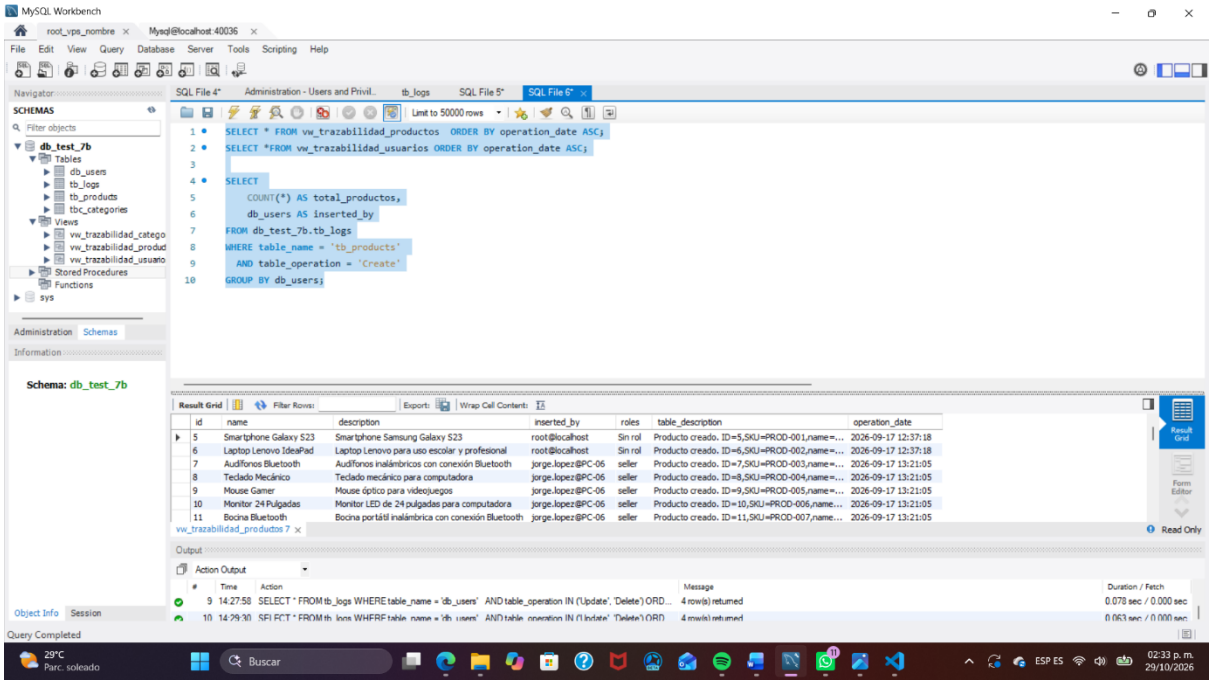


6.Usuario Support elimina y modifica un usuario

7.Usuario root inserta 2 productos



8.Seller 1 inserta 5 productos



9.Seller 2 inserta 5000 productos

